



OS Fundamentals Overview

Brief Overview

This note covers **Operating Systems** and was created from a 161-page PDF of the textbook. It covers **process management**, **memory handling**, I/O design, security, and real-world case studies.

Key Points

- Overview of OS structure and architecture
- In-depth look at process and memory concepts
- Examination of I/O and storage management
- Review of protection, security, and real-world OS case studies



Overview of the Book

- **Target audience:** junior/senior undergraduates or first-year graduate students; also useful for practitioners.
- **Prerequisites:** basic data structures, computer organization, a high-level language (C or Java).
- **Approach:** intuitive descriptions, figures and examples replace formal proofs; examples drawn from Linux, Windows, macOS, Android, iOS, Solaris, BSD.



Operating System Fundamentals

Definition – *An operating system is a program that manages a computer's hardware, provides a basis for application programs, and acts as an intermediary between the computer user and the computer.*

- **Primary goals**
 - Provide a convenient, efficient environment for users to execute programs.
 - Optimize utilization of hardware resources (CPU, memory, I/O).
 - Protect the system and its resources from unauthorized or erroneous use.



Book Structure (Six Major Parts)

Part	Main Topics
Part I – Overview	What operating systems do; computer-system organization; OS architecture; open-source and computing environments.
Part II – Process Management	Process concept, scheduling, interprocess communication (IPC), threads, synchronization, deadlock overview.
Part III – Memory Management	Main memory, swapping, paging, segmentation, virtual allocation, demand paging, copy-on-write, thrashing.

Part IV – Storage Management	Mass-storage structure, RAID, disk scheduling, file-system interface & implementation, I/O subsystem, performance issues.
Part V – Protection & Security	Protection mechanisms, access matrix, authentication, cryptography, firewalling, security policies.
Part VI – Case Studies	Detailed studies of Linux, FreeBSD, Mach; additional material available on the supporting website.

Process Management (Chapters 3-6)

- **Process concept** – unit of work; includes user-level and OS-level processes.
- **Process operations** – creation, termination, state transitions.
- **Interprocess Communication (IPC)** – mechanisms for processes to exchange data.
- **Scheduling** – algorithms, criteria, real-time considerations, multiprocessor scheduling.
- **Threads** – parallelism, Amdahl's law, implicit threading (OpenMP, Grand Central Dispatch).
- **Synchronization** – mutex locks, semaphores, monitors, classic problems (e.g., producer-consumer).
- **Deadlock** – overview presented in Chapter 6 (previously a full chapter).

Memory Management (Chapters 7-8)

- **Main memory management** – allocation strategies, swapping (including Intel 32- and 64-bit, ARM).
- **Paging** – page tables, contiguous memory, segmentation.
- **Virtual allocation** – demand paging, copy-on-write, thrashing, replacement algorithms.
- **Kernel memory allocators** – SLUB, SLOB (Linux).

Storage Management (Chapters 9-12)

- **Mass-storage structure** – disks, RAID levels, swap space.
- **Disk scheduling** – algorithms for ordering I/O requests.
- **File-system interface** – file concepts, access methods, directories, mounting.
- **File-system implementation** – allocation methods, free-space management, efficiency.
- **I/O subsystem** – hardware overview, synchronous vs. asynchronous, blocking vs. non-blocking, vectored I/O, performance metrics.

Protection & Security (Chapters 13-14)

- **Protection mechanisms** – access matrix, capability-based protection, language-based protection, revocation of rights.
- **Security fundamentals** – confidentiality, integrity, authentication, cryptography, firewalling, threat categories (program, system, network).
- **Implementation notes** – OS-level enforcement, secure kernel design, example of Windows 7 security.

Case Studies (Chapter 15 & Appendices)

- **Linux** – history, design principles, kernel modules, process and memory management, scheduling, file systems.
- **FreeBSD** – UNIX heritage, system structures.

- **Mach** – microkernel concepts.

Programming Environments

Environment	Key Features	Example Use
POSIX	Portable OS interface; core standard (POSIX.1); extensions: real-time (POSIX.1b), threads (POSIX.1c).	C programs illustrating system calls, Pthreads, real-time APIs.
Java	Built-in thread support; platform-independent VM.	Java 1.6 programs for concurrency and networking.
Windows API	Comprehensive set of functions for processes, threads, memory, devices.	C programs targeting Windows XP/7.

- Most examples are in **C**; some in **Java**.
- The book supplies a **Linux virtual machine** with gcc, editors, and kernel-module assignments.

Supporting Website (<http://www.os-book.com>)

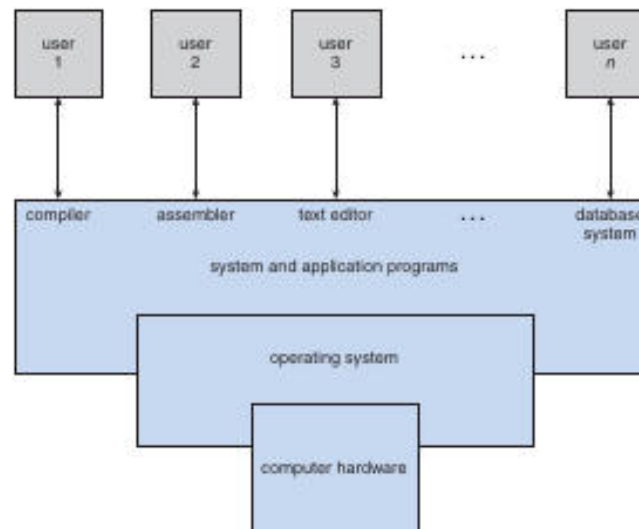
- Linux virtual machine download.
- Source code (C & Java).
- Sample syllabi, PowerPoint slides, figures, review questions with solutions.
- FreeBSD and Mach case-study materials.
- Study guide, errata, and additional exercises.

✨ Edition Changes (Second Edition vs. Ninth Edition)

Change	Description
Removed Chapters	7 (Deadlocks) – now an overview; 16 (Virtual Machines); 17 (Distributed Systems); 19 (Windows 7); 20 (Influential Systems).
Added / Updated Topics	Multicore systems, mobile computing (iOS, Android, OS X), cloud computing, real-time overview, kernel data structures, Linux CFS scheduler, Windows user-mode scheduling, ARM architecture, solid-state disks, vectored I/O, updated cryptography section.
Reorganized	Process synchronization moved before scheduling; real-time scheduling integrated; storage-management chapters renumbered.
New Sections	Kernel data structures, hybrid mobile-device interfaces, Amdahl's law, implicit threading (Grand Central Dispatch), mutex-lock extensions, encryption updates.

System Organization Overview

- A computer system can be divided into four major components: **users**, **application programs**, the **operating system (OS)**, and **hardware** (CPU, memory, I/O devices).
- The OS sits between users/applications and hardware, coordinating access to resources.



The diagram shows layers from users and applications (compilers, editors, databases) down through the OS to the underlying hardware, illustrating the clear separation of responsibilities.

User View of the OS

From the user's perspective, the operating system is a "government" that provides a convenient environment for running programs.

- **Single-user systems** – desktop or laptop PCs where the OS optimizes for one person's workload.
- **Multi-user systems** – mainframes or minicomputers accessed via terminals; the OS must share CPU, memory, and I/O fairly among many users.
- **Workstation/networked environments** – users have dedicated workstations but still share resources such as file and print servers.
- **Mobile devices** – smartphones and tablets typically operate as standalone users, often with touch-screen interfaces instead of keyboard/mouse.
- **Embedded/automotive devices** – may have minimal or no direct user interaction; they run autonomously to perform specific functions.

The primary goal in each case is **resource utilization**: maximize useful work (or play) while keeping performance acceptable for the intended user scenario.

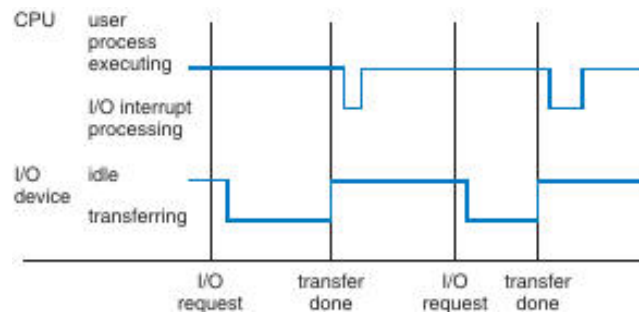
System View of the OS

From the system's perspective, the OS is a resource manager and control program.

- Manages **CPU time**, **memory**, **storage space**, and **I/O devices**.
- Must allocate resources **fairly** and **efficiently**, especially when multiple processes or users compete.
- Provides a **control interface** to prevent erroneous or unsafe operations (e.g., illegal memory accesses, device misuse).

System Startup (Boot Process)

- The **bootstrap program** resides in read-only memory (ROM or EEPROM) and runs immediately after power-on or reset.
- It performs hardware initialization (CPU registers, controllers, memory) and then **locates the OS kernel**, loading it into main memory.
- Once the kernel starts, it creates the first user-level process (on UNIX this is init), which in turn launches other system daemons and services.



The timeline illustrates how the CPU switches between executing user processes and handling I/O interrupts during normal operation.

Interrupt Handling

An interrupt is a signal that temporarily halts the CPU's current activity to service a device or event.

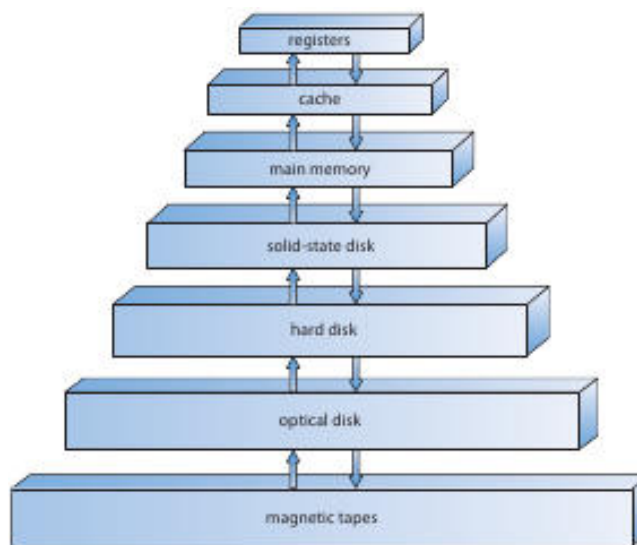
- The CPU saves its current state, jumps to a **fixed entry point**, and executes an **interrupt service routine (ISR)**.
- To speed up dispatch, most architectures use an **interrupt vector table**: an array of pointers indexed by device number that directly points to the appropriate ISR.
- After the ISR finishes, the saved CPU state is restored and execution resumes as if the interrupt never occurred.

Storage Structure Basics

- **Bit** – the smallest unit, representing 0 or 1.
- **Byte** – 8 bits; the typical addressable unit in most machines.
- **Word** – architecture-dependent (e.g., 64-bit word = 8 bytes).

Storage Hierarchy

Level (top → bottom)	Typical Technology	Volatility	Typical Size	Speed
Registers	CPU-internal	Volatile	Few bytes	Fastest
Cache	SRAM	Volatile	KB–MB	Very fast
Main Memory	DRAM	Volatile	GB	Fast
Solid-State Disk (SSD)	NAND flash / DRAM + battery backup	Non-volatile	Hundreds of GB	Fast
Hard Disk Drive (HDD)	Magnetic platters	Non-volatile	TB	Moderate
Optical Disk	CD/DVD/Blu-ray	Non-volatile	GB	Slow
Magnetic Tape	Reel-to-reel	Non-volatile	TB–PB	Slowest



The pyramid visualizes the trade-off between speed, cost, and volatility across storage levels.

- **Volatile** storage (top four levels) loses contents when power is removed; **non-volatile** storage (SSD and below) retains data.
- Some SSDs combine DRAM speed with a battery-backed backup to protect data during power loss.
- **Flash memory** is a common SSD type used in cameras, PDAs, and increasingly in general-purpose computers.
- **NVRAM** (DRAM with battery backup) offers DRAM-level speed while being non-volatile as long as the battery holds charge.

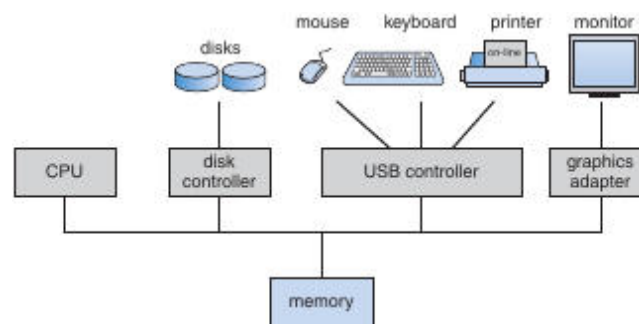
Designing a memory system requires balancing **expensive fast memory** (registers, cache) with **cheaper large-capacity storage** (HDD, tape).

I/O Structure and Device Management

- A **device controller** handles a specific class of peripherals (e.g., disks, audio, video).
- The OS supplies a **device driver** that translates generic OS requests into controller-specific commands, presenting a uniform interface to the rest of the system.

I/O Operation Flow

1. **Driver** loads command parameters into the controller's registers.
 2. **Controller** moves data between its local buffer and the peripheral.
 3. Upon completion, the controller raises an **interrupt**.
 4. **ISR** notifies the driver, which may return data or status to the requesting process.
- **Interrupt-driven I/O** is efficient for small transfers but incurs high overhead for bulk data.
 - **Direct Memory Access (DMA)** allows the controller to transfer large blocks directly between its buffer and main memory without CPU intervention; only a single interrupt signals completion.



The diagram shows the CPU, memory, and various controllers (disk, USB, graphics) linked to peripheral devices, illustrating where drivers, interrupts, and DMA fit into the overall architecture.

☒ Processor Architectures

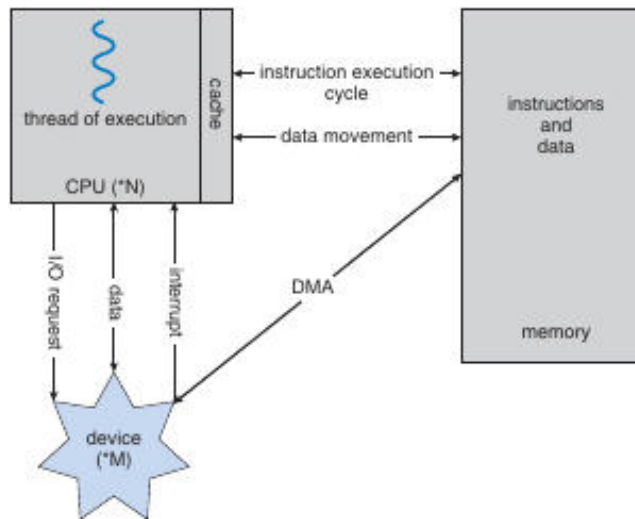
1.3.1 Single-Processor Systems

- One general-purpose CPU executes user instructions.
- May include **special-purpose processors** (e.g., disk controllers, graphics processors) that operate independently of the main CPU.
- These auxiliary processors handle limited instruction sets and are often managed by the OS for tasks such as disk scheduling.

1.3.2 Multiprocessor (Multicore) Systems

- Two or more CPUs share a common bus, memory, and I/O subsystem.
- **Advantages:**
 1. **Increased throughput** – more work can be performed simultaneously.
 2. **Improved responsiveness** – background tasks can run on separate cores while the foreground task stays responsive.
 3. **Scalability** – additional cores can be added to meet growing performance demands.

- Modern desktops, laptops, servers, and even smartphones now commonly employ multicore designs.



The flowchart depicts how the CPU interacts with memory and devices via instruction execution, DMA, parallel I/O, and interrupts, highlighting the coordination required in both single- and multiprocessor environments.

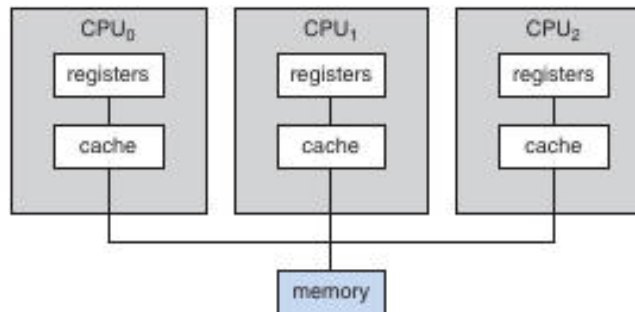
Multiprocessor Systems Overview

Multiprocessor system – a computer architecture that contains two or more CPUs that share common resources such as main memory and I/O devices.

Types of Multiprocessing

Type	Key Characteristics	Typical Use
Asymmetric Multiprocessing (AMP)	One <i>boss</i> processor controls system; workers have predefined tasks or request instructions from the boss.	Early mainframes, some embedded controllers.
Symmetric Multiprocessing (SMP)	All CPUs are <i>peers</i> ; each can run user and OS processes.	Modern desktops, servers, Unix-like OSes (e.g., AIX).
Multicore	Multiple cores placed on a single silicon die; each core has its own registers and cache, optionally sharing higher-level caches.	Contemporary CPUs (Intel, AMD, ARM).
Clustered Systems	Several independent nodes (each a full computer) connected via a fast LAN or SAN; loosely coupled but appear as a single system.	High-availability services, high-performance computing.

Blade Servers	Processor, I/O, and networking boards mounted in a chassis; each blade may run its own OS, some are SMP nodes.	Data-center consolidation, scalable cloud services.
----------------------	--	---



The diagram shows three CPUs (CPU0-CPU2) each with private registers and cache, all connected to a single shared memory module, illustrating the classic SMP layout.

Advantages and Challenges

- **Speed-up** – Adding CPUs can increase throughput, but **Amdahl's law** tells us that overhead (synchronization, contention) reduces the ideal linear gain.
- **Economy of scale** – Shared power, mass, and storage lower total cost compared with many single-processor machines.
- **Reliability** – Failure of one processor degrades performance (e.g., 10% loss with 10 CPUs) rather than causing a total crash.
 - *Graceful degradation* and *fault tolerance* allow the system to keep running; HP NonStop uses lock-step CPU pairs that compare results and restart the faulty pair on another CPU set.

Memory Access Models

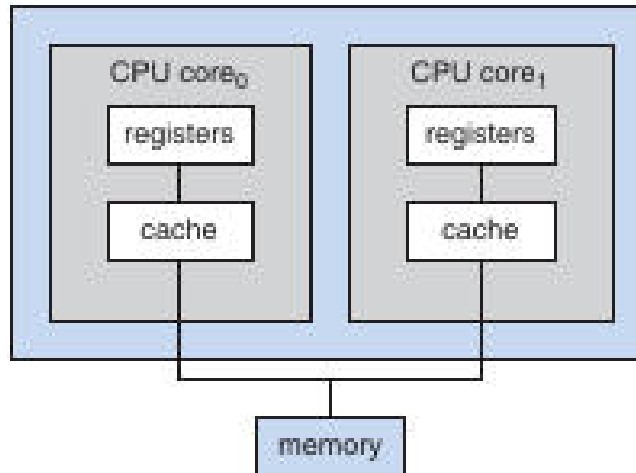
Uniform Memory Access (UMA) – every CPU accesses any location in main memory with the same latency.

Non-Uniform Memory Access (NUMA) – memory is divided into regions; a CPU accesses its “local” region faster than remote regions, potentially incurring performance penalties.

Operating systems mitigate NUMA penalties through intelligent resource placement and scheduling (see Chapter 8.5.4).

Multicore Design Details

- Each core possesses **registers** and **L1 cache**; higher-level caches may be private or shared.
- On-chip communication is far faster than inter-chip links, reducing latency and power consumption.



Two cores (core0, core1) each contain registers and cache, sharing a common main-memory interface.

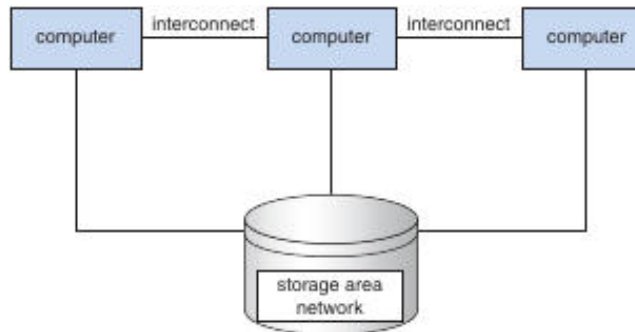
Clustered Systems

Cluster – a set of network-connected computers that share storage and cooperate to provide a single, highly available service.

High-Availability Clustering

Style	Description	Pros	Cons
Asymmetric (active-standby)	One node runs the application; another waits in hot-standby to take over on failure.	Simple fail-over logic.	Under-utilizes standby hardware.
Symmetric (active-active)	All nodes run the application and monitor each other.	Full utilization of resources.	Requires coordination to avoid conflicts.
Parallel clusters	Nodes share a common storage system; applications are explicitly written to run in parallel across nodes.	Massive performance scaling.	Demands special OS support and locking mechanisms.

Beowulf clusters are a low-cost variant built from commodity PCs running Linux and open-source communication libraries. They are popular for scientific computing where thousands of inexpensive nodes deliver petaflop-scale performance.



Three computers connect to a shared SAN, illustrating how clustered nodes can access a common disk pool for high-availability and parallel I/O.

Multiprogramming & Time-Sharing

Multiprogramming – keeping several jobs in main memory simultaneously so the CPU can always find a ready process, thereby improving utilization.

Time-sharing – a logical extension of multiprogramming where the CPU switches among processes so rapidly that each user perceives exclusive use of the system.

How Multiprogramming Works

1. **Job pool** – all processes awaiting execution reside on disk.
2. **Ready set** – a subset of jobs currently loaded into RAM.
3. **CPU dispatch** – the OS selects a ready process; when it blocks for I/O, the CPU is given to another ready process.
4. **Swapping** – if memory becomes full, the OS may swap out a paused process to make room for a new one.

Time-Sharing Characteristics

- **Interactive I/O** – keyboard, mouse, touchscreen inputs are slow; the OS masks this latency by pre-emptively scheduling other processes.
 - **Response time goal** – typically < 1 second per user request.
 - **Scheduling & memory management** – both are essential; see Chapters 6 (scheduling) and 7-8 (memory).
-

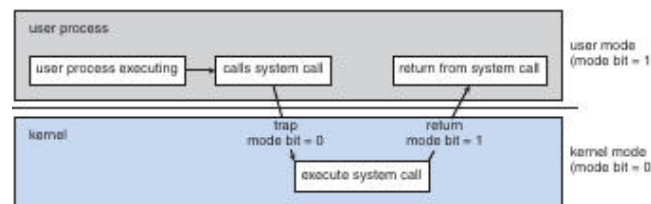
Protection Mechanisms: Dual-Mode Operation

Dual-mode operation – the hardware provides a **mode bit** that distinguishes **kernel (supervisor) mode** (0) from **user mode** (1). Privileged instructions may only execute in kernel mode.

Mode Transition Flow

1. **User process** runs in user mode.
2. When it needs OS services, it issues a **system call** (software interrupt).

3. The CPU switches to kernel mode, jumps to the interrupt service routine.
4. Kernel validates parameters, performs the service, then restores user mode.



The flowchart shows the transition from user mode to kernel mode during a system call, and the return to user mode after service completion.

Privileged Instructions (examples)

- I/O control operations
- Timer management
- Interrupt enable/disable
- Changing page tables

If a user program attempts a privileged instruction, the hardware generates a **trap** and the OS terminates or handles the fault.

Extensions Beyond Two Modes

- Some CPUs add a **virtual-machine mode** for hypervisors (more privileged than user mode but less than kernel).
- Additional privilege levels (e.g., x86 rings) allow finer-grained control for kernel subsystems.



Timer Interrupts for Preemptive Multitasking

Timer interrupt – a periodic hardware signal that forces the CPU to enter kernel mode, enabling the OS to regain control and possibly pre-empt a running process.

Timer Mechanics

- A **counter** is programmed with a tick count (e.g., 1 ms).
- A fixed-rate **clock** decrements the counter on each tick.
- When the counter reaches zero, the hardware raises an interrupt; the OS's timer ISR runs, resets the counter, and may invoke the scheduler.

Typical configurations:

- 10-bit counter → intervals from 1 ms up to 1024 ms in 1 ms steps.
- Fixed-rate clocks (e.g., 60 Hz for classic display refresh) are also used for periodic tasks.

Timer interrupts are the foundation of **preemptive multitasking**, ensuring that no user process can monopolize the CPU (e.g., by looping forever).



Timer Interrupts & Preemptive Multitasking

Timer interrupt – a periodic hardware signal that forces the CPU into kernel mode so the OS can regain control of the processor.

- The OS programs a **counter** with the maximum CPU time a user program may consume (e.g., 7 minutes → 420 seconds).
- Each timer tick (typically 1 s) decrements the counter.
- While the counter > 0 the OS returns control to the user process; when it reaches 0 the OS terminates the program.
- The timer's control registers are **privileged**; only the kernel may modify them, preventing a user program from disabling preemption.



Process Concept (expanding on earlier definition)

Process – an executing program together with its allocated resources (CPU time, memory, I/O devices) and state information.

- **Resources needed:**
 1. **CPU time** – scheduled by the OS.
 2. **Memory space** – code, data, stack.
 3. **I/O devices** – files, printers, network sockets.
- A *program* is passive code stored on disk; a *process* is the active incarnation of that program.
- Processes may **create subprocesses** via system calls (e.g., fork/exec on UNIX).
- When a process ends, the OS reclaims all its resources.



Memory Management Fundamentals

Main memory – a large, byte-addressable array that holds both instructions and data while a program runs.

- Directly addressable by the CPU; all I/O must ultimately pass through memory.
- Because only a fraction of a system's workload can reside in memory at once, the OS must **manage memory**:

OS responsibility	Description
Allocation tracking	Keep a record of which memory regions are in use.
Swapping / paging	Decide which pages to bring into memory or evict to secondary storage.
Dynamic allocation	Allocate and free blocks as processes request or release them.
Protection	Ensure a process can only access its own address space.



Storage Management Overview

Storage management – the OS’s provision of a logical, device-independent view of persistent data.

Media type	Typical characteristics
Hard disk (HDD)	Magnetic, large capacity, moderate speed, random access.
Solid-state disk (SSD)	NAND flash, fast random access, non-volatile.
Optical (CD/DVD/Blu-ray)	Sequential/ random, lower speed, removable.
Magnetic tape	Very high capacity, sequential, archival.

- The OS abstracts these devices as **files** and **directories**.
- Core storage-management duties:
 - Create / delete files and directories.
 - Map logical files onto physical blocks.
 - Maintain free-space information.
 - Perform **disk scheduling** to order I/O requests efficiently.
 - Provide backup/archival services.

File-System Management Details

File – a collection of related information identified by a name, stored on a storage medium.

- Files may contain **programs**, **documents**, **binary data**, etc.
- Organized into **directories** (folders) to form a hierarchical namespace.
- **Permissions** control which users or groups may read, write, or execute a file (e.g., UNIX mode bits, Windows ACLs).
- The OS enforces these permissions whenever a process issues a file-system call.

Caching Principles

Cache – a small, fast storage area that holds copies of data expected to be reused soon.

Level	Typical size	Access time	Managed by
Registers	< 1 KB	0.25-0.5 ns	Compiler / hardware
L1/L2 Cache	16-64 KB	0.5-25 ns	Hardware
Main memory	1-64 GB	80-250 ns	OS
SSD / HDD	TBs	10 ⁴ -10 ⁵ ns	OS (file-system cache)

- **Software-controlled caches** (e.g., page cache) are kept coherent by the OS; **hardware caches** (CPU caches) rely on hardware coherence protocols.
- In multiprocessor systems, **cache-coherency** ensures that updates made by one CPU become visible to others promptly.

I/O Subsystem Architecture

I/O subsystem – the collection of components that hide device-specific details from the rest of the OS.

- **Components:**
 1. **Memory-management layer** – buffering, spooling, caching.
 2. **Device-driver interface** – generic API exposed to the kernel.
 3. **Device drivers** – translate generic requests into controller-specific commands.
- **Interrupt-driven I/O:** device raises an interrupt; the ISR notifies the driver, which wakes the waiting process.
- **DMA** (Direct Memory Access) lets a controller move large blocks between its buffer and main memory without CPU intervention, reducing overhead.

Protection & Security Essentials

Protection – mechanisms that regulate access to system resources.

- **Dual-mode operation** (kernel vs. user) prevents user code from executing privileged instructions.
- **Authentication** identifies a user (e.g., username/password).
- **Authorization** (access control lists, capabilities) determines what the authenticated user may do.
- **Security policies** (confidentiality, integrity, availability) guide the design of OS safeguards such as firewalls and encryption.

Kernel Data Structures

Lists, Stacks, & Queues

- **Singly linked list** – each node points to its successor; end marked by null.
- **Doubly linked list** – nodes have forward and backward links, enabling traversal in both directions.
- **Circular list** – last node links back to the first, eliminating null termination.
- **Stack** – LIFO structure; operations: push (add) and pop (remove). Used for function call frames.
- **Queue** – FIFO structure; useful for scheduling ready processes, printer jobs, etc.

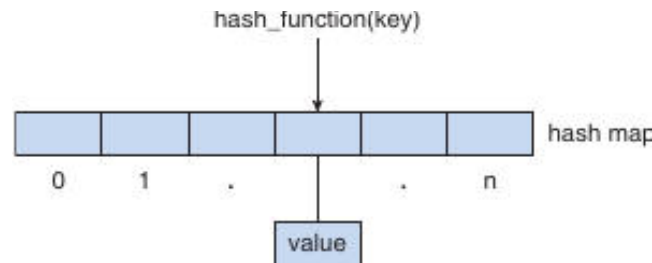
Trees

- **General tree** – parent may have any number of children.
- **Binary tree** – each node has at most two children (left $\leq$ right).
- **Binary search tree (BST)** – left subtree $\leq$ node $\leq$ right subtree; enables $O(\log n)$ search when balanced.
- OS kernels often employ **balanced BSTs** (e.g., red-black trees) for fast look-ups in scheduling and memory-allocation algorithms.

Hash Functions & Maps

Hash function – a deterministic algorithm that maps a key to an index in a table.

- Enables average-case $O(1)$ retrieval of key-value pairs.
- **Collisions** (different keys mapping to the same slot) are handled by chaining (linked list per bucket) or open addressing.



The diagram shows a key being processed by `hash_function(key)`, which selects a bucket in the hash table where the associated value is stored.

Bitmaps

Bitmap – a compact array of bits where each bit represents the state (free/used) of a resource.

- Ideal for tracking large numbers of uniform resources (e.g., disk blocks).
- Example: 001011101 → bits 0, 1, 3, 4 are free; bits 2, 5, 6, 7 are allocated.
- Space-efficient: one bit per resource versus a full byte or larger data type.

The above sections build on earlier concepts of process scheduling, memory allocation, and system security, providing a cohesive view of how modern operating systems manage resources, protect assets, and organize internal data.

Mobile Computing 🖐️

Mobile computing – the use of handheld smartphones and tablet devices that are portable, lightweight, and typically have limited screen size, memory, and processing power compared with desktops.

- **Key characteristics**
 - **Portability** – devices are carried and used on the go.
 - **Wireless connectivity** – IEEE 802.11 Wi-Fi, cellular data (3G/4G/5G).
 - **Rich sensors** – GPS for location, accelerometer/gyroscope for orientation and motion detection, camera, microphone.
- **Typical applications**
 - Email, web browsing, multimedia (music, video, e-books).
 - Sensor-driven services (navigation, fitness tracking, augmented-reality overlays).
 - Real-time interaction (gaming that reacts to device tilt or shaking).

Example: An accelerometer-enabled game lets the player control a character by tilting the tablet rather than using a mouse or keyboard.

- **Resource constraints**
 - Storage often ≤ 64 GB (e.g., smartphones) vs. many terabytes on desktops.
 - Processors are smaller and slower to conserve power; battery life is a primary design concern.
- **Dominant mobile operating systems**

- **iOS** – runs on Apple iPhone, iPad, and Apple TV.
- **Android** – powers a wide range of smartphones and tablets from many manufacturers.

(Chapter 2 provides deeper coverage of iOS and Android internals.)

Distributed Systems

Distributed system – a collection of physically separate computers that are network-connected to provide users with shared resources, increased computation speed, and higher availability.

- **Motivations**
 - **Scalability** – add more nodes to handle larger workloads.
 - **Reliability** – failure of a single node degrades performance rather than causing a total crash.
 - **Geographic distribution** – resources can be placed close to users, reducing latency.

Network Foundations

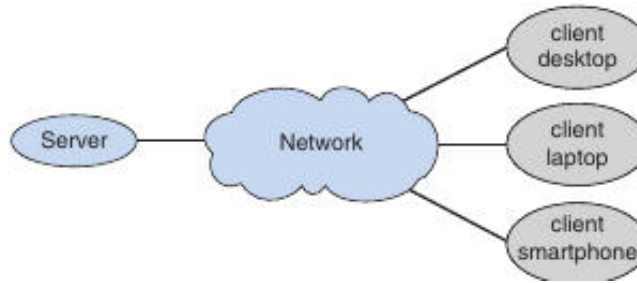
Network type	Typical scope	Example use
LAN (Local-Area Network)	Single building or campus	Office file sharing
MAN (Metropolitan-Area Network)	City-wide	Municipal Wi-Fi
WAN (Wide-Area Network)	Inter-city or global	Corporate branch connectivity
PAN (Personal-Area Network)	Few-foot radius (Bluetooth, 802.11)	Phone ↔ headset

- **Common protocol** – **TCP/IP**, the backbone of the Internet; supported by virtually all general-purpose OSes.
- **Device driver role** – each network interface (Ethernet, Wi-Fi, cellular) requires a driver that exposes a uniform API to the OS.

Client-Server Computing

Client-server model – a distributed architecture where **clients** request services and **servers** provide those services over a network.

- **Typical server categories**
 - **Compute server** – receives a request, performs a computation (e.g., database query), returns the result.
 - **File server** – offers a file-system interface for create/read/update/delete operations; often serves web pages to browsers.



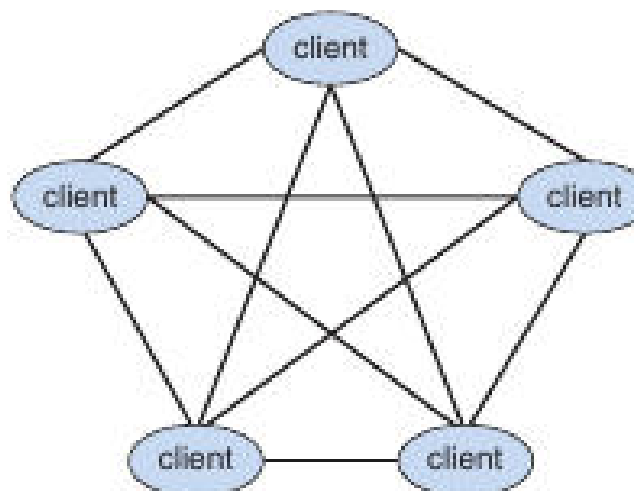
The diagram shows a central server connected via a network cloud to three client devices: a desktop, a laptop, and a smartphone. It illustrates how diverse client platforms can access the same server resources.

- **Advantages over centralized terminals**
 - Clients run locally, reducing load on the server.
 - Richer user interfaces (graphical, touch) are possible on the client side.
 - Easier scaling: add more servers to handle increased demand.

🔗 Peer-to-Peer (P2P) Computing ⚡

Peer-to-peer – a distributed architecture where every node can act as both client and server, offering services directly to other peers.

- **Two discovery models**
 1. **Centralized lookup** – a node registers its service with a dedicated directory; other peers query the directory to locate the provider.
 2. **Pure broadcast** – a node broadcasts a service request; any peer that can fulfill it replies directly.



The image depicts five fully interconnected peers, illustrating the mesh connectivity typical of P2P systems.

- **Historical examples**
 - **Napster** – used a central index of files stored on peers.

- **Gnutella** – relied on broadcast queries without a central index.
- **Skype** – combines a login server (centralized) with decentralized voice-over-IP (VoIP) streams.
- **Benefits**
 - No single bottleneck; bandwidth and storage are aggregated across all participants.
 - Resilience: the system can survive loss of many nodes.
- **Legal considerations** – many early P2P networks faced copyright infringement lawsuits (e.g., Napster shut down in 2001).

Virtualization & Emulation

Virtualization – the technique that allows multiple guest operating systems to run concurrently on a single physical machine, each appearing to have its own hardware.

- **Key components**
 - **Virtual Machine Manager (VMM)** – also called a hypervisor; mediates access to physical resources (CPU, memory, I/O).
 - **Guest OS** – runs inside a VM, unaware that the underlying hardware is shared.

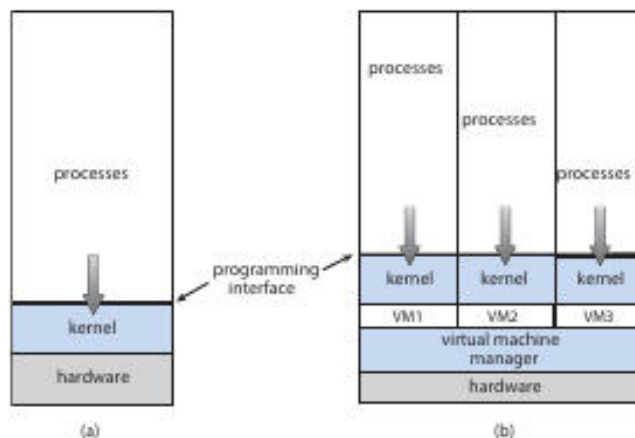


Diagram (a) shows a classic three-layer OS (processes → kernel → hardware). Diagram (b) adds a VMM between the hardware and multiple kernels, illustrating how virtual machines isolate guests.

- **Emulation vs. virtualization**
 - **Emulation** translates instructions from a source CPU architecture to a target architecture (e.g., Apple Rosetta translating PowerPC code to Intel x86). This incurs a large performance penalty.
 - **Virtualization** runs the guest OS on the *same* CPU architecture, allowing near-native performance; only privileged operations are intercepted by the VMM.
- **Typical uses**
 - Development and testing across multiple OSes on one workstation.
 - Server consolidation: multiple services run on separate VMs on a single physical server.
 - Cloud providers: thousands of VMs hosted on large clusters (see Cloud Computing below).

☁ Cloud Computing ☀☁

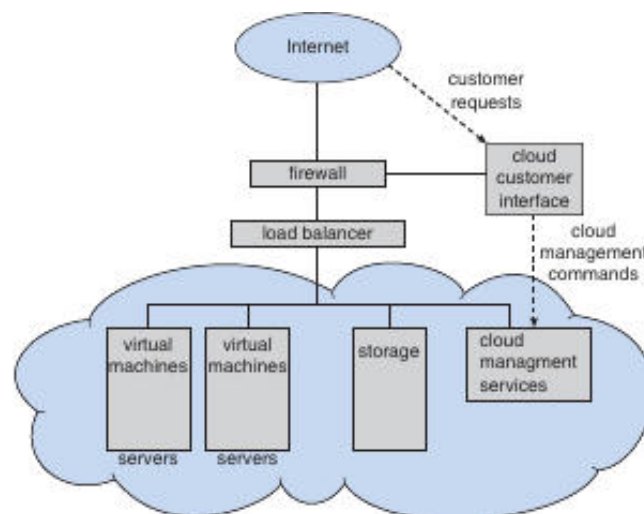
Cloud computing – delivery of computing resources (servers, storage, applications) as services over a network, typically the Internet, built on top of virtualization.

- **Service models**

Model	What is provided
SaaS (Software as a Service)	End-user applications (e.g., web-based email, spreadsheets).
PaaS (Platform as a Service)	Development stack (runtime, database, middleware) for building custom apps.
IaaS (Infrastructure as a Service)	Virtual machines, storage, networking primitives (e.g., Amazon EC2).

- **Deployment models**

- **Public cloud** – services offered to anyone over the Internet (pay-as-you-go).
- **Private cloud** – operated by a single organization for its own use.
- **Hybrid cloud** – combination of public and private resources, allowing workload bursting.



The diagram shows Internet → firewall → load balancer → cloud containing virtual machines, storage, and management services, illustrating the typical IaaS stack.

- **Management tools**

- Proprietary (e.g., VMware Director) and open-source (e.g., Eucalyptus) platforms automate VM provisioning, scaling, and monitoring.

- **Relation to OS concepts**

- The hypervisor acts as the low-level OS for guests; higher-level cloud services add orchestration, billing, and multi-tenant isolation.

Embedded & Real-Time Systems

Embedded system – a computer integrated into a larger device that performs dedicated functions, often with limited user interaction.

- **Common domains**
 - Automotive control units, industrial robots, consumer appliances (microwaves, DVD players), medical devices.
- **Operating-system traits**
 - Small footprint, minimal UI, often a **real-time operating system (RTOS)** when strict timing guarantees are required.

Real-time system – an OS that must complete critical operations within predefined deadlines; missing a deadline can cause system failure.

- **Typical RTOS responsibilities**
 - Deterministic scheduling (priority-based, often pre-emptive).
 - Minimal interrupt latency.
 - Predictable memory allocation (avoid dynamic fragmentation).
 - **Examples of RTOS use**
 - Engine fuel-injection controllers, weapon-system guidance, scientific-experiment data acquisition.
 - **Contrast with time-sharing systems**
 - Time-sharing aims for fairness and high throughput; RTOS prioritizes deadline compliance over overall throughput.
-

Open-Source Operating Systems 🙌

History

- Early computing (1950s) featured **open-source-style sharing**: hobbyist groups (e.g., MIT's Tech Model Railroad Club) exchanged source listings.
- Commercial pressures later drove many vendors to distribute **binary-only** products, using licensing and DRM to restrict copying.
- The **Free Software Movement** (Stallman, 1983) introduced the **GNU Project** and the **GPL**, demanding that redistributed binaries be accompanied by source code and that modifications remain under the same license.

Linux Overview

- **Linux kernel** – originally released by Linus Torvalds in 1991; written in C, licensed under GPL-2.0.
- Combined with GNU tools (compilers, utilities) to form a complete **GNU/Linux** operating system.
- **Distributions** package the kernel, libraries, and applications into coherent systems (e.g., Red Hat Enterprise, Ubuntu, Debian, SUSE, Slackware).
 - Some target **enterprise** environments (Red Hat Enterprise Linux).

- Others are **Live** media that run directly from CD/DVD/USB without installation (PCLinuxOS, “SuperGamer”).
 - **Virtualization support** – modern Linux includes built-in hypervisors (KVM), containerization (Docker, LXC), and extensive driver frameworks, making it a popular base for cloud infrastructures.
 - **Learning advantage** – full source availability lets students read, modify, and re-compile the kernel, providing hands-on insight into OS internals (process scheduling, memory management, device drivers, etc.).
-



Additional Implementation Detail

- The Linux kernel provides a **balanced search-tree** implementation using **red-black trees** (see header `linux/rbtree.h`). This data structure underpins many kernel subsystems (e.g., timer management, memory-area tracking) due to its guaranteed $O(\log n)$ insertion, deletion, and lookup while maintaining a relatively simple implementation.



Open-Source Operating Systems as Learning Platforms

Open-source OS – an operating system whose complete source code is freely available for inspection, modification, and redistribution.

- Virtual-machine “appliance” images (e.g., from VMware Appliances) let students **boot a fully configured Linux, BSD, or Solaris system** without installing on real hardware.
- The same image can be reused across labs, enabling hands-on exploration of kernel code, system calls, and driver interfaces.

Linux → Debian Virtual Machine

- Download a pre-installed Debian image, boot it with **VMware Player** (free).
- The image contains the full Linux source tree; source resides under `/usr/src/`.
- Throughout the textbook, examples reference this Debian VM.

BSD UNIX

BSD UNIX – a family of UNIX-like systems derived from the University of California, Berkeley’s extensions to AT&T’s System V.

- First released in **1978**; source and binary distributions were openly shared, but AT&T licensing made early versions non-free.
- A lawsuit delayed progress until **4.4BSD-Lite** (1994) became the first fully open release.
- Major modern BSD variants: **FreeBSD**, **NetBSD**, **OpenBSD**, **DragonFlyBSD**.

Exploring FreeBSD

- Obtain a FreeBSD VM image, boot it as described for Linux.
- Kernel source lives in `/usr/src/sys`; memory-management code is under `/usr/src/sys/vm`.

macOS Darwin

Darwin – the open-source core of Apple’s macOS, built around the XNU kernel.

- Source releases are posted at <http://www.opensource.apple.com/> for every macOS version.
- The kernel package name always begins with “xnu.”
- Apple’s developer portal (<http://connect.apple.com>) supplies documentation and support.

Solaris → OpenSolaris → Illumos

- **Solaris** originated as Sun Microsystems’ commercial UNIX (based on AT&T System V).
- In **2005** Sun open-sourced much of Solaris as **OpenSolaris** (available at <http://src.opensolaris.org/source>).
- After Oracle’s acquisition of Sun (2009) the project’s future became uncertain, but the code base lives on through **Project Illumos** (<http://wiki.illumos.org>), which extends OpenSolaris with new features.



Open-Source Projects as Educational Tools

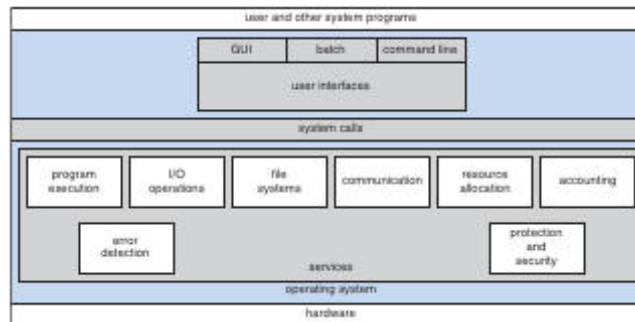
Free-software movement – a cultural and legal initiative that promotes the unrestricted sharing and modification of software.

- Thousands of projects (e.g., kernels, compilers, utilities) are indexed on sites such as **Freshmeat** and **DistroWatch**.
- Benefits for students:
 - **Read** the actual implementation of OS concepts (scheduling, memory management, file systems).
 - **Modify** code to experiment with alternative algorithms.
 - **Debug** real-world bugs, gaining experience comparable to professional development.
 - Access to historic systems (e.g., **Multics**) for architectural study.
- Licensing interactions (GPL, BSD, MIT) sometimes limit cross-pollination, yet many components (e.g., parts of OpenSolaris) have been **ported to BSD** and vice-versa, illustrating practical interoperability.



Core Operating-System Services (Figure 2.1)

Operating-system service – a function provided by the kernel or system libraries that abstracts hardware complexities for applications and users.



The diagram shows four layers: (1) user programs & interfaces (GUI, batch, CLI), (2) system calls, (3) OS services (process management, protection, security), and (4) hardware.

1 User Interface

- **Command-line interface (CLI)** – textual commands entered via keyboard.
- **Batch interface** – commands stored in script files and executed sequentially.
- **Graphical user interface (GUI)** – windows, icons, menus, and pointing devices.

2 Program Execution

- Load executable into memory, create a **process**, and start execution.
- Handles normal termination and abnormal events (e.g., segmentation fault).

3 I/O Operations

- Provides abstract system calls for devices (disk, network, printer).
- Users cannot issue raw I/O instructions; the OS mediates access for safety and efficiency.

4 File-System Manipulation

- Create, delete, rename, and change permissions of files/directories.
- Supports multiple file-system types with differing performance and reliability characteristics.

5 Interprocess Communication (IPC)

- **Shared memory** – multiple processes map the same physical pages.
- **Message passing** – send/receive packets via kernel-mediated queues.

6 Error Detection & Handling

- Detect hardware faults (memory errors, I/O failures) and software exceptions (illegal memory access, arithmetic overflow).
- Typical responses: halt, terminate offending process, or return error codes.

7 Resource Allocation

- **CPU scheduling** – decide which ready process receives the next time slice.
- **Memory management** – allocate/deallocate pages, enforce protection.
- **Device management** – assign I/O devices, manage buffers.

8 Accounting

- Record per-user or per-process consumption of CPU time, memory, and I/O for billing or statistical analysis.

9 Protection & Security

- Enforce **access control** (ACLs, capability lists).
- Authenticate users (passwords, tokens) and isolate processes via **dual-mode operation** (kernel vs. user).

 User ↔ OS Interface: Command Interpreters (Shells)

Command interpreter (shell) – a program that reads user commands, parses them, and invokes the appropriate system services.

- Some systems embed the interpreter in the kernel; others run it as a regular user-level process launched at login.
- **Shells** can be swapped out; UNIX/Linux offers **Bourne (sh)**, **Bourne-Again (bash)**, **Korn (ksh)**, and many third-party shells.

Internal vs. External Commands

1. **Internal commands** – implemented directly inside the shell (e.g., cd, exit).
 - Faster because no separate program load is needed.
2. **External commands** – separate executable files located via the PATH environment variable (e.g., rm, grep).
 - The shell forks a new process, loads the program, and uses a system call (e.g., execve) to run it.

Example workflow:

User types rm file.txt → shell searches PATH → forks → child process calls execve("/bin/rm", ...) → kernel loads /bin/rm and executes it → upon completion, the child exits and the shell returns to the prompt.

 Summary of Key Concepts (Section 6)

Concept	Why It Matters for Students
Virtual-machine images	Provide a reproducible, sandboxed environment for OS experimentation.
BSD lineage	Illustrates how open-source forks evolve (FreeBSD, NetBSD, OpenBSD).
Darwin/XNU	Shows an open-source kernel embedded in a commercial OS (macOS).
Illumos	Demonstrates community-driven continuation of a formerly commercial code base.
OS services list	Serves as a checklist when mapping textbook theory to real-world system calls.

Shell design	Highlights the separation between user interaction and kernel execution.
Open-source learning	Enables hands-on modification of core OS components, reinforcing theoretical concepts.

These notes build directly on the earlier sections covering **process management, memory hierarchy, and multiprocessor architectures**, providing the concrete platforms (Linux, BSD, Solaris) where those abstract ideas are implemented.

User & Interface Operating-System

Command-Line Interface (Shell)

A shell is a command-interpreter program that reads user input, parses it, and invokes the appropriate system services.

- **Internal commands** – built into the shell (e.g., cd, exit). Faster because no separate program load is needed.
- **External commands** – separate executable files located via the PATH (e.g., rm, ls).
 - The shell **forks** a child process, the child **execve()**s the external program, then the kernel loads the binary and runs it.
- Adding new commands is as simple as creating a properly named executable file in a directory that is on the PATH.
- Example from the text: the rm command removes files; its functionality is defined completely by the code in its executable file.

Graphical User Interfaces (GUI)

A GUI lets users interact with the OS by manipulating visual objects (windows, icons, menus) with a pointing device.

- **Desktop metaphor** – windows represent programs/files, icons act as shortcuts, menus group commands.
- **Historical milestones**
 1. Xerox **Alto** (1973) – early research prototype.
 2. Apple **Macintosh** – popularized the desktop with the **Aqua** interface in macOS X.
 3. Microsoft **Windows 1.0** – added a GUI on top of MS-DOS; later versions refined both appearance and functionality.
- **Mobile touchscreens** replace the mouse with gestures (tap, swipe, pinch).
 - The iPad example illustrates a full-screen touch UI where the keyboard is simulated on the display.

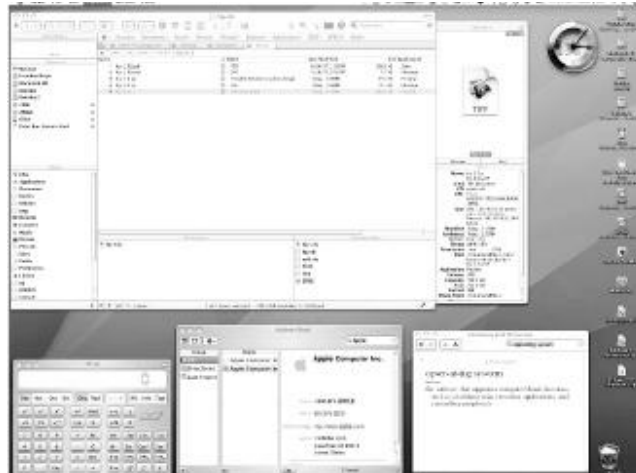


The iPad home screen shows icons (apps) arranged on a beach-background wallpaper, demonstrating how touch-based GUIs present programs as visual objects.

- **Unix-like GUI environments** – CDE, X-Windows, KDE, GNOME (all open-source).

Choice of Interface

- **System administrators** often prefer the CLI because:
 - It enables rapid, scriptable automation of repetitive tasks.
 - Commands can be recorded in shell scripts and re-executed without recompilation.
- **General users** tend to favor GUIs for ease of discovery and visual feedback.
- Some OSes provide **both**: e.g., macOS X offers the Aqua GUI *and* a Unix-style command line (Terminal).

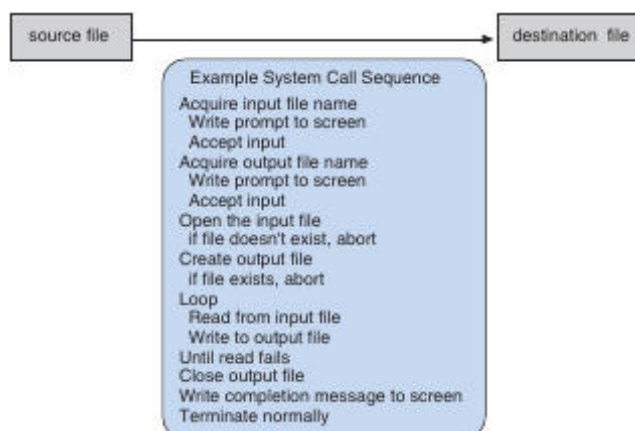


Typical Windows desktop with multiple overlapping windows and a taskbar, illustrating the GUI paradigm.

☎ System Calls

System calls are the programmed interface through which user applications request services from the kernel.

- System calls are usually **wrapped** by a language-specific library (e.g., libc in C) that presents a more convenient API.
- **Example workflow** – copying a file:
 1. Prompt user for source and destination names (output & input system calls).
 2. open() source for reading; open() destination for writing.
 3. Loop: read() a block → write() the block.
 4. Handle errors (e.g., end-of-file, disk full) via return codes.
 5. close() both files, terminate.
- Even this simple program may invoke **dozens of system calls**, illustrating how heavily applications rely on the OS kernel.



The flowchart lists the steps of a typical file-copy program, showing the succession of system calls required.

Standard APIs

API	Primary platforms	Typical functions
POSIX	UNIX, Linux, macOS	open(), read(), write(), fork(), execve()
Windows API	Windows	CreateProcess(), ReadFile(), WriteFile(), CreateFile()
Java API	All JVM platforms	FileInputStream, FileOutputStream, ProcessBuilder

- Example: the POSIX read() prototype

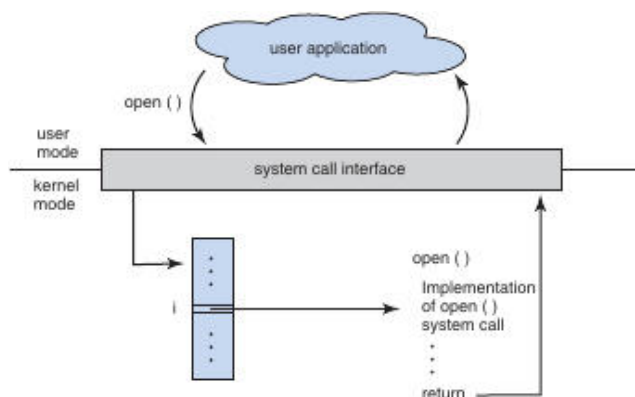
```
#include
ssize_t read(int fd, void *buf, size_t count);
```

- fd – file descriptor
- buf – buffer to receive data
- count – maximum bytes to read
- Returns number of bytes read (0=EOF, -1=error).
- Using the API abstracts the actual system-call number and calling convention, enhancing **portability** across OS variants.

Passing Parameters to System Calls

Three common mechanisms:

1. **Registers** – fastest, but limited by the number of argument registers.
2. **Memory table** – a block in user space holds all arguments; the address of the block is passed in a register (used by Linux, Solaris).
3. **Stack** – arguments are pushed onto the user stack; the kernel reads them from there.



The diagram shows the user program placing arguments in a table, passing the table's address in a register, and the kernel extracting the parameters during the system-call.

Types of System Calls

System calls are grouped into six major categories (Figure 2.8 in the textbook).

Category	Typical Operations
Process Control	fork(), exec(), exit(), wait(), getpid(), setpriority()
File Management	create(), delete(), open(), close(), read(), write(), lseek(), stat()
Device Management	request_device(), release_device(), read(), write(), ioctl(), getdevinfo()
Information Maintenance	gettimeofday(), settimeofday(), uname(), sysinfo()
Communications	socket(), bind(), listen(), accept(), send(), recv()
Protection	chmod(), chown(), setuid(), setgid(), access()

Process Control Examples

- fork() creates a child process; both parent and child continue execution.
- execve() replaces the current image with a new program.
- wait() blocks the parent until a child terminates, returning the child's exit status.

File Management Examples

- open() obtains a file descriptor.
- read() / write() transfer data between the file descriptor and user buffers.
- close() releases the descriptor.

Device Management Example

- Many OSes treat devices as special files; open("/dev/sda", ...) yields a descriptor that can be used with read()/write() just like a regular file.

Communication Example

- socket() creates an endpoint; connect() or bind() + listen() + accept() establish a communication channel.

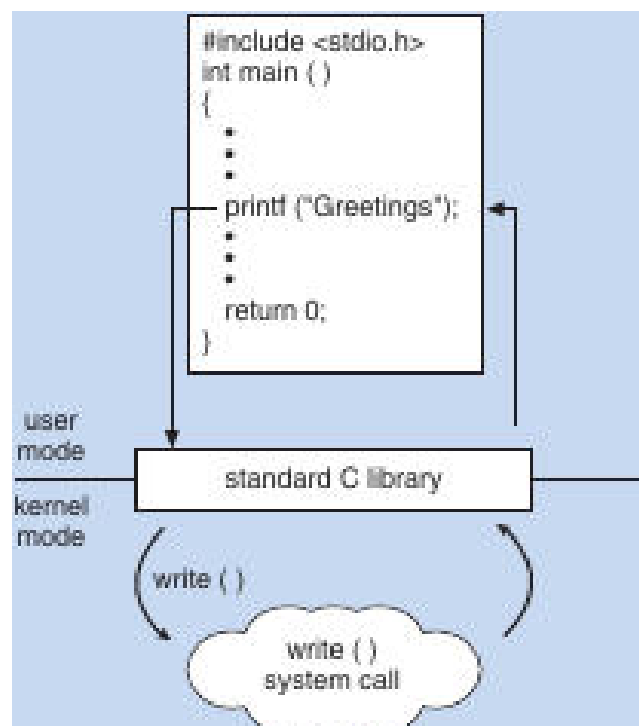
Windows vs. Unix System Calls

Functionality	Windows API (example)	Unix / Linux API (example)
Create Process	CreateProcess()	fork() + execve()
Terminate Process	ExitProcess()	exit()

Wait for Event	WaitForSingleObject()	waitpid()
File Operations	CreateFile(), ReadFile(), WriteFile(), CloseHandle()	open(), read(), write(), close()
Device I/O	SetConsoleMode(), ReadConsole(), WriteConsole()	ioctl()
Time / Sleep	SetTimer(), Sleep()	alarm(), sleep()
Inter-process Communication	CreatePipe(), CreateFileMapping(), MapViewOfFile()	pipe(), shm_open(), mmap()
Security / Protection	SetFileSecurity(), InitializeSecurityDescriptor()	chmod(), chown(), setuid()

Example: C Program Calling printf()

The printf() library function ultimately invokes the **write()** system call to send characters to the standard output.



The diagram shows a user-mode C program calling printf(), which uses the standard C library to issue a write() system call that executes in kernel mode.

- **User mode** → printf() (in libc) → kernel mode write() → OS writes bytes to the terminal device.

Summary of Key Points

- **Shells** provide a programmable CLI; external commands are regular executables discovered via PATH.
- **GUIs** rely on windows, icons, and pointing devices; they evolved from Xerox PARC to modern touch-based mobile interfaces.
- **System calls** are the privileged gateway to kernel services; libraries expose them via portable APIs.
- Parameters can be passed via **registers**, **memory tables**, or the **stack**, each with different limits.
- System calls fall into six functional categories, covering everything from process creation to security.
- Windows and Unix expose analogous functionality through different naming conventions but often map to similar underlying kernel actions.

Information Maintenance (System Calls)

Information-maintenance system calls – calls whose primary purpose is to retrieve or modify system-wide data such as time, user lists, OS version, memory or disk usage.

- Typical examples: `time()`, `date()`, `getuid()`, `gethostname()`, `getrusage()`.
- Debug-oriented calls: `dump()` (memory dump), single-step trap support, profiling via timer interrupts.
- Profiling: a timer interrupt records a counter each tick; frequent interrupts give a fine-grained picture of where the program spends time.

Common categories

Category	Example calls	Typical use
Clock & date	<code>time()</code> , <code>date()</code>	Obtain current time/date for logging, time-stamping.
System status	<code>getuid()</code> , <code>getpid()</code> , <code>gethostname()</code> , <code>sysinfo()</code>	Query users, process IDs, host identity, memory/disk statistics.
Debug / profiling	<code>dump()</code> , <code>ptrace()</code> , timer-interrupt based profiling	Generate core dumps, step through code, measure execution hotspots.
Process attributes	<code>getpriority()</code> , <code>setpriority()</code> , <code>getrlimit()</code> , <code>setrlimit()</code>	Inspect or change scheduling priority, resource limits.

These calls complement the **process-control** and **file-management** calls described earlier.

Communication (Interprocess Communication)

Interprocess communication (IPC) – mechanisms that allow two or more processes to exchange data or synchronize actions.

Two dominant IPC models are supported by modern OSes:

Message-Passing Model

- Processes exchange **messages** either **directly** (known endpoints) or **indirectly** via a **mailbox**.

- A connection must be opened first; the communicating parties must know each other's **process identifier (PID)** or **network host name**.
- Typical workflow:
 1. **Server** (often a daemon) calls `accept()` and blocks waiting for a client.
 2. **Client** calls `connect()` (or `open_connection()`) using the server's address.
 3. Both sides use `read()` / `write()` (or `message()`) to transfer data.
 4. When finished, `close_connection()` terminates the session.
- System calls involved: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `read()`, `write()`, `close()`.

Shared-Memory Model

- Processes map a common memory region into their address spaces using `shmat()/shmget()` (or `attach()` on some OSes).
- The OS still enforces **protection**; only processes granted the appropriate permissions may attach.
- Synchronization (to avoid simultaneous writes) is handled by higher-level primitives such as **mutexes**, **semaphores**, or **monitor** constructs (see Chapter 5).

Both models coexist in most systems; message passing excels for small, well-structured data, while shared memory offers the highest transfer speed for large data sets.

Protection

Protection – OS mechanisms that control access to hardware resources, files, memory regions, and privileged operations.

Key system calls (often grouped in a **permission** API):

Call	Purpose
<code>set_permission()</code> / <code>get_permission()</code>	Manipulate access rights for files, devices, or other objects.
<code>allow_user()</code> / <code>deny_user()</code>	Grant or revoke a specific user's right to a resource.
<code>chmod()</code> , <code>chown()</code> (POSIX)	Change mode bits or ownership of files/directories.
<code>setuid()</code> , <code>setgid()</code>	Change the effective user/group ID of a process (privilege escalation or reduction).

Protection was once a concern only for multi-user mainframes, but with networking, mobile devices, and cloud services it now applies to **all** computing platforms.

System Programs

System programs (sometimes called **system utilities**) provide a convenient environment for development, administration, and everyday tasks. They can be grouped as follows:

Category	Typical examples
----------	------------------

File management	cp, mv, rm, ls, mkdir, tar
Status information	date, uptime, free, df, top
File modification	vi, emacs, sed, awk
Programming-language support	gcc, clang, perl, python, gdb
Program loading & execution	ld.so (dynamic linker), execve wrapper scripts
Communications	ssh, ftp, mail, telnet, ping
Background services (daemons)	cron, syslogd, sshd, printd

Most utilities are thin wrappers around system calls; some (e.g., gcc) are complex programs that invoke many kernel services.

Operating-System Design & Implementation

Design Goals

Design goals – high-level specifications that guide OS architecture.

Goal type	Desired property
User-oriented	Convenience, learnability, reliability, safety, speed.
System-oriented	Flexibility, reliability, efficiency, maintainability.

Different environments (embedded real-time, general-purpose desktops, high-performance clusters) prioritize these goals differently.

Mechanisms vs. Policies

Mechanism – the underlying code that performs a function (e.g., timer interrupt, page-fault handler).

Policy – the rule that decides *what* the mechanism should do (e.g., how long a time slice is, which process gets priority).

Separating the two allows a single mechanism to support many policies without code changes.

- Example: a **scheduler** mechanism can be driven by a **time-sharing**, **real-time**, or **fair-share** policy simply by loading a different configuration table.
- Microkernel designs push most policies into user-space services, keeping the kernel mechanism minimal.

Implementation Languages

Layer	Typical language(s)
Hardware-level kernel core	Assembly (for bootstrapping, context save/restore)

Core kernel services	C (occasionally C++)
Device drivers	C (some assembly for performance-critical paths)
System utilities & daemons	C, C++, scripting languages (Perl, Python, shell)
User-space applications	Any language supported by the platform

Higher-level languages improve portability and developer productivity; performance-critical sections can still be hand-optimized in assembly when needed.

OS Structural Approaches

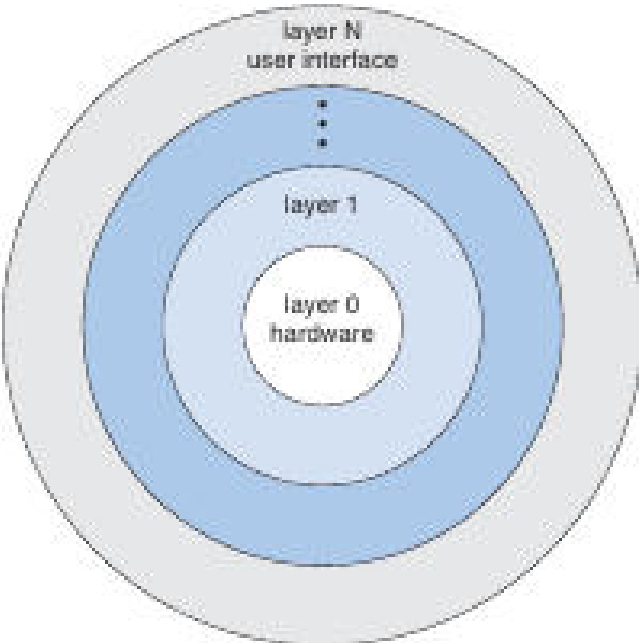
1 Simple (Monolithic) Structure

- Early systems like **MS-DOS** and the first **UNIX** kernels placed most functionality in a single address space.
- Advantages: minimal call-overhead, high performance.
- Disadvantages: poor modularity, limited protection, difficult to maintain or extend.

2 Layered Approach

Layered OS – a hierarchy where each layer only uses services of the layer directly below it.

- **Layer 0** = hardware.
- **Layer N** = user interface.



The concentric-circle diagram illustrates layers from hardware (inner) up to the user interface (outer). Each layer

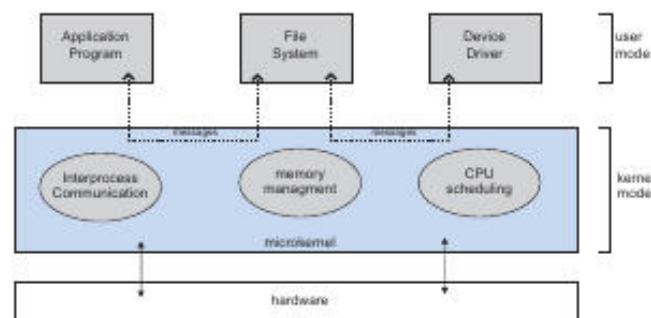
provides services to the one above and relies only on the layer below.

- Benefits: easier debugging, clear interfaces, incremental verification.
- Drawbacks: possible performance penalty because each system call may traverse multiple layers.

3 Microkernel

Microkernel – a minimal kernel that implements only essential mechanisms (process & memory management, IPC) while most services run in user space.

- Communication between user-level services and the microkernel occurs via **message passing**.
- Example services: file system, device drivers, network stack.



The diagram shows the microkernel core providing IPC, memory management, and CPU scheduling. User-level services such as the file system and device drivers interact with the kernel only through messages.

- Pros: high modularity, easier fault isolation, portability.
- Cons: higher overhead due to frequent IPC; historically slower than monolithic kernels (e.g., early Windows NT).

4 Loadable Kernel Modules (LKMs)

Loadable kernel modules – pieces of code that can be linked into the running kernel at boot time or on-the-fly.

- Core kernel provides a stable API; additional functionality (new filesystems, device drivers, scheduling algorithms) is added as modules.
- Combines benefits of layered design (clear interfaces) with flexibility of microkernel-style dynamic extension.

Typical module categories (illustrated for Solaris):

Module class	Examples
Core kernel	CPU scheduler, memory manager
Device drivers	Disk, network, USB
File-system modules	UFS, ZFS, NFS client
Stream modules	Network protocol stacks

Executable formats	ELF loader, a.out support
--------------------	---------------------------

Loadable modules enable rapid prototyping and reduce the need to rebuild the entire kernel for minor extensions.

Summary of Core Concepts

- **Information-maintenance** calls expose system state; profiling uses timer interrupts.
- **IPC** offers two models: message passing (socket-style) and shared memory (fast, but needs synchronization).
- **Protection** is enforced via permission-related system calls; it is universal across all modern platforms.
- **System programs** range from simple file utilities to complex compilers and daemons; they are thin wrappers over kernel services.
- **Design goals** balance user convenience with system robustness; separating **mechanisms** from **policies** yields flexible, adaptable OSes.
- **Implementation languages** progress from assembly (bootstrapping) to C/C++ for the kernel, and higher-level languages for utilities.
- **Structural approaches** evolve from monolithic (early DOS/UNIX) to layered, microkernel, and finally to loadable-module designs, each trading off performance, modularity, and ease of maintenance.

Hybrid Operating-System Structures

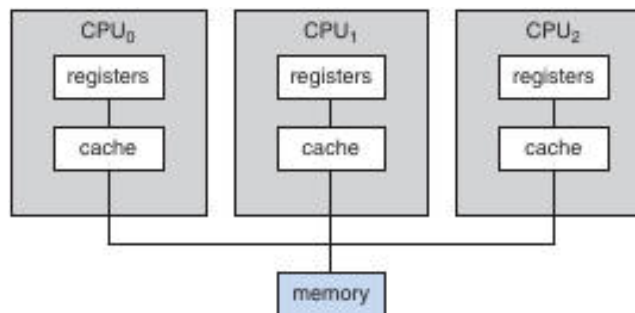
Hybrid OS – an operating-system architecture that combines elements of monolithic, modular, and microkernel designs to balance performance, extensibility, and reliability.

OS	Core kernel style	Loadable modules	Notable subsystems
Linux	Monolithic (single address space)	Loadable kernel modules (device drivers, filesystems, etc.)	Device-driver modules, LKM for new filesystems, networking
Solaris	Monolithic kernel with a core kernel plus seven LKM categories (scheduling classes, file systems, loadable system calls, executable formats, STREAMS modules, miscellaneous, device & bus drivers)	Yes – modules add functionality without recompiling the kernel	Same as Linux, plus explicit <i>loadable system-call</i> modules
Windows	Largely monolithic for performance	Supports dynamically loadable kernel extensions (drivers) and separate user-mode subsystems (e.g., networking, graphics)	Subsystems run as user-mode processes, providing OS-personality isolation

macOS X	Hybrid – microkernel Mach + BSD UNIX kernel (layered)	Kernel extensions (kexts) are loadable at runtime	Aqua UI, Cocoa (Objective-C API), Mach IPC & thread scheduler, BSD POSIX layer, device-driver framework
iOS	Based on macOS X kernel (Mach + BSD) with added mobile-specific layers	Same kext mechanism; additional Cocoa Touch frameworks	Media Services (graphics/audio/video), Core Services, Cocoa Touch (Objective-C UI), I/O Kit
Android	Linux kernel (modified)	Loadable drivers via standard Linux LKM mechanism	Dalvik (now ART) VM, core libraries (libc, SQLite, WebKit, OpenGL), Application Framework, hardware abstraction layer (HAL)

macOS X Architecture

1. **Aqua UI** – top-most user interface layer.
2. **Cocoa** – Objective-C API for application development.
3. **Mach microkernel** – provides memory management, RPC, IPC, and thread scheduling.
4. **BSD kernel** – supplies POSIX APIs, traditional UNIX command line, and file-system support.
5. **Kernel extensions (kexts)** – dynamically loadable modules for device drivers and other services.



The diagram shows the layered organization of macOS X, from the Aqua UI down through Mach, BSD, and kernel extensions.

iOS Stack

- **Cocoa Touch** – Objective-C frameworks tailored for touch input and mobile hardware.
- **Media Services** – graphics, audio, and video pipelines.
- **Core Services** – networking, security, and system-wide utilities.
- **BSD layer** – provides the underlying POSIX-compatible kernel services.

Android Stack

- **Linux kernel** (customized by Google) – handles process, memory, and device-driver duties.
- **Core Libraries** – libc, SQLite, WebKit, OpenGL.

- **Dalvik/ART VM** – executes bytecode compiled from Java source; optimized for limited CPU power.
 - **Application Framework** – high-level APIs for UI, telephony, location, etc.
 - **Hardware Abstraction Layer (HAL)** – isolates the kernel from hardware specifics.
-



Debugging & Performance Tuning

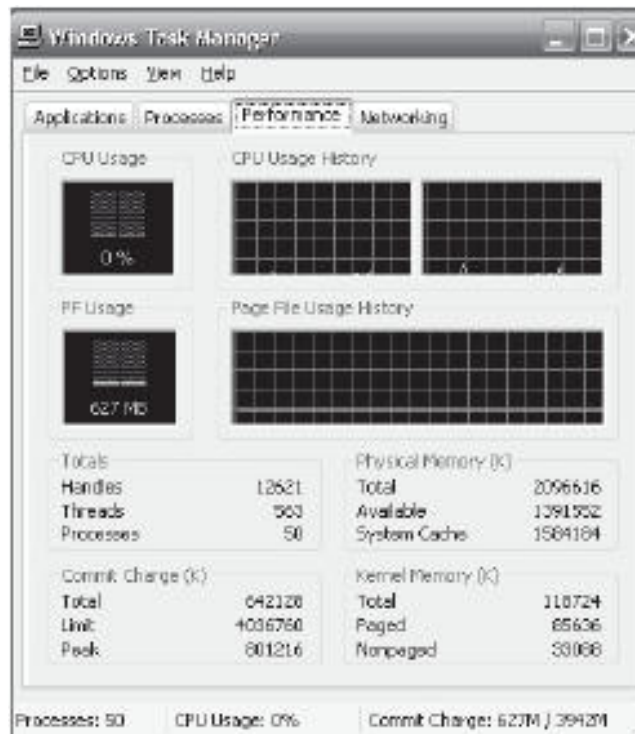
Debugging – the systematic process of locating, diagnosing, and fixing faults in both hardware and software components of a system.

Failure Analysis & Core Dumps

- When a process crashes, most OSes write an **error log** and optionally a **core dump** – a snapshot of the process’s memory image saved to a file.
- Kernel crashes (e.g., file-system panic) are handled similarly, but the dump is written to a **reserved, non-filesystem area** to avoid reliance on corrupted storage.
- Post-mortem analysis: a debugger loads the core dump, allowing inspection of registers, stack frames, and variable values.

Performance-Tuning Techniques

Technique	Typical Tool	What It Shows
System-wide tracing	top (UNIX), Task Manager (Windows)	Real-time CPU, memory, I/O usage per process
Log-based tracing	Kernel trace logs, custom trace files	Time-stamped events (scheduler switches, I/O completions)
Profiling	gprof, perf, DTrace scripts	Function-level execution time, call frequencies



The Windows Task Manager screenshot displays CPU and memory graphs, process counts, and detailed resource statistics—useful for spotting bottlenecks.

DTrace: Dynamic Tracing Facility

DTrace – a low-overhead, safe, dynamic tracing framework (originating in Solaris 10) that can instrument both user-mode and kernel-mode code without requiring prior instrumentation.

Core Concepts

Term	Role
Provider	Component that defines a set of related probes (e.g., syscall, io, sched).
Probe	A hook inserted at a specific location (function entry, system call, I/O event).
Consumer	User-level program that enables probes, defines actions, and receives data.
ECB (Enabled Control Block)	Stores a predicate and a list of actions for a particular probe; multiple ECBs can fire on the same probe.
Safety Compiler	Compiles D scripts to bytecode guaranteed not to crash or hang the kernel.

Example DTrace Script (Scheduler-time accounting)


```
sched:::on-cpu /uid == 101/ { self->ts = timestamp; }
sched:::off-cpu /self->ts/ { @time[execname] = sum(timestamp - self->ts); }
```

- **sched:::on-cpu** fires when a thread starts executing on a CPU.
- **self->ts** stores the entry timestamp in thread-local storage.
- **sched:::off-cpu** fires on exit; the difference yields CPU time for that thread.
- The aggregation `@time[execname] = sum(...)` produces total nanoseconds per executable.

Running the script (`dtrace -s sched.d`) yields output similar to:

```
gnome-settings-daemon    142354
gnome-vfs-daemon         158243
...
gnome-terminal           5007269
java                     10769137
```

This demonstrates **per-process CPU consumption** for a specific user ID, all gathered without stopping the system.

Safety & Performance Guarantees

- Probes are **disabled** when not in use, incurring virtually no overhead.
- When enabled, the probe inserts a tiny call to `dtrace_probe()`; the framework ensures the call is cheap and does not block.
- Execution time of probe actions is limited; exceeding thresholds causes automatic disabling of the offending probe.
- Buffers are allocated per-CPU to avoid contention and data loss.

Adoption Beyond Solaris

- Ported to **OpenSolaris**, **FreeBSD**, and **macOS**.
- Linux research projects (e.g., **SystemTap**, **eBPF**) provide comparable capabilities, inspired by DTrace's model.



System Generation (SYSGEN)

System generation – the process of tailoring an operating system to a specific hardware configuration, producing a customized kernel and supporting software.

Information Gathered During SYSGEN

1. **CPU characteristics** – model, instruction-set extensions, number of cores.
2. **Memory layout** – total RAM, addressable range (detected by probing until an illegal-address fault).
3. **Boot-disk geometry** – partition scheme, filesystem types, block size.
4. **Device inventory** – device numbers, interrupt lines, driver requirements.
5. **Desired OS options** – max processes, scheduling policies, security features.

Generation Approaches

Approach	When Modules Are Linked	Advantages	Disadvantages
Compile-time selection	Only needed modules are compiled into the kernel image.	Small, efficient binary; no unused code.	Requires recompilation for every configuration change.
Link-time selection	A pre-compiled library of modules is linked based on a configuration file.	Faster than full recompilation; still relatively compact.	Slightly larger binaries; still needs a build step.
Table-driven (run-time)	All code resides in the kernel; a configuration table selects which parts are active.	No recompilation needed; quick adaptation to hardware changes.	Larger memory footprint; potential performance impact from unused code.

Typical SYSGEN Workflow

1. **Probe hardware** – the SYSGEN program queries the machine (CPU ID, memory size, attached devices).
2. **Create configuration description** – either interactively entered by an administrator or auto-generated from probes.
3. **Select modules / set parameters** – based on the description, the generator includes the appropriate drivers, filesystems, and kernel options.
4. **Build kernel image** – compile/link or assemble tables, producing the final OS binary.
5. **Deploy** – write the image to the boot medium (disk, ROM, flash).

The resulting system can be **re-generated** when hardware is added or removed, ensuring optimal resource usage and minimal attack surface.

 System Boot Recap (for Continuity)

Booting – the sequence that loads the operating-system kernel into memory and transfers control from firmware to the kernel.

- **Bootstrap program** (stored in ROM/EPROM or on a boot disk) performs early diagnostics, initializes critical hardware, and loads the kernel image.
- On PCs, the **GRUB** bootloader reads a boot block (often block0) and hands over control to the kernel located elsewhere on disk.
- After the kernel starts, it creates the initial user-level process (init on UNIX-like systems) which in turn launches system daemons and user sessions.

Refer to the earlier “System Startup (Boot Process)” section for a detailed step-by-step description.

 Summary of Core Concepts

- **Hybrid OS designs** blend monolithic speed with modular extensibility; macOS X, iOS, and Android illustrate layered hybrids built on a microkernel or customized Linux core.
- **Debugging** relies on logs, core dumps, and dynamic tracing; DTrace provides safe, on-the-fly instrumentation for both user and kernel code.

- **Performance tuning** uses system-wide tracing (top, Task Manager) and fine-grained probes (DTrace) to locate bottlenecks.
- **System generation** tailors an OS to specific hardware by probing resources, selecting modules, and producing a customized kernel image.
- **Boot process** transitions control from firmware to the kernel, after which the OS creates the first user process and proceeds with normal operation.



Interprocess Communication (IPC) Models

IPC is the set of mechanisms that enable processes to exchange data or synchronize their actions.

1 Message-Passing Model

- **How it works:** Processes send and receive discrete messages through system calls such as send(), recv(), write(), read(), or via sockets.
- **Strengths**
 - Natural fit for distributed systems; messages can travel across network boundaries.
 - Strong encapsulation – each process keeps its own address space, reducing accidental memory corruption.
- **Weaknesses**
 - Overhead of copying data between user buffers and kernel buffers.
 - Limited bandwidth for large data transfers compared with shared memory.

2 Shared-Memory Model

- **How it works:** Two or more processes map the same physical pages into their virtual address spaces (e.g., shmget(), mmap()).
- **Strengths**
 - Very fast data exchange; no copying required after the initial mapping.
 - Ideal for large data structures (e.g., multimedia buffers).
- **Weaknesses**
 - Requires explicit synchronization (mutexes, semaphores) to avoid race conditions.
 - More difficult to reason about correctness; bugs can corrupt shared data.



Mechanism vs. Policy Separation

Separating mechanism (the “how”) from policy (the “what”) makes an OS more flexible and easier to evolve.

- **Mechanism** – low-level code that performs a primitive operation (e.g., timer interrupt, page-fault handler, context switch).
- **Policy** – decision logic that uses the mechanism (e.g., length of a time slice, page-replacement algorithm).
- **Desirability**
 - Enables **plug-and-play** of different policies without rewriting kernel code.
 - Facilitates **experimentation** (research OS can swap scheduling policies while keeping the same timer mechanism).
 - Improves **maintainability**; bugs in a mechanism affect all policies, but a faulty policy can be changed locally.

Layered Design & Tight Coupling

A clean layered OS places each component above another, using only the layer-below's services.

- When two components **depend on each other**, a strict layering becomes ambiguous.
- **Scenario requiring tight coupling**: a **real-time scheduler** that must inspect hardware-specific timer registers *and* the process control block simultaneously. The scheduler needs direct access to both the hardware timer (low-level) and the ready-queue structures (high-level), making a strict separation impractical.

Microkernel Architecture

Main Advantage

- **Minimal kernel** – only core mechanisms (CPU scheduling, low-level IPC, basic memory management) run in privileged mode, reducing the trusted code base and improving reliability.

Interaction Model

- **User-space servers** provide higher-level services (filesystems, network stacks, device drivers).
- Applications request a service → **IPC** to the appropriate server → server may invoke the kernel's minimal mechanisms and return results.

Disadvantages

- **Performance overhead** due to frequent IPC and context switches between user-space servers and the kernel.
- **Complexity in service design**; ensuring atomicity across multiple servers can be difficult.
- Historically, **larger code footprints** for user-space servers can offset the kernel's small size.

Loadable Kernel Modules (LKMs)

- **Runtime extensibility** – new functionality (drivers, filesystems) can be inserted without rebooting.
- **Reduced kernel rebuilds** – developers compile only the module, not the entire kernel tree.
- **Isolation of faults** – a buggy module can be unloaded (rmmod) while the rest of the kernel continues operating.
- **Facilitates experimentation** – researchers can test new algorithms (e.g., a novel scheduler) as a module before committing to a permanent kernel change.

Mobile OS Comparison: iOS vs. Android

Aspect	iOS	Android
Kernel	XNU (Mach microkernel + BSD components)	Modified Linux kernel
Runtime	Objective-C/Swift with Cocoa Touch framework	Java-style bytecode executed by Dalvik/ART VM

App Packaging	.ipa bundles, signed by Apple	.apk packages, signed by developer
Permission Model	Strict sandbox per app; runtime entitlements	Permission declared in AndroidManifest.xml
Update Model	Centralized via App Store; OS updates controlled by Apple	OTA updates via Google/OEM; many device-specific forks
Hardware Integration	Tight integration with Apple hardware (Secure Enclave)	Broad hardware support; HAL abstracts vendor specifics

Both platforms share a layered architecture (UI → framework → kernel) and rely on sandboxing for security, but they differ in kernel lineage, programming languages, and distribution models.



Why Android Java Programs Do Not Use the Standard Java API/VM

- Android uses **Dalvik/ART**, a custom VM that executes **DEX** bytecode, not standard **JVM** bytecode.
- The **standard Java API** (e.g., java.awt, java.swing) depends on desktop-oriented libraries absent on mobile devices.
- Android supplies its own **framework APIs** (android.*) optimized for touch input, low memory, and power management.
- Using a custom VM lets Google **control memory layout**, **garbage-collection pauses**, and **security sandboxing** tailored for mobile constraints.



Synthesis OS Kernel Design

Synthesis embeds frequently used system-call routines directly in kernel space as hand-written assembler, shortening the path a call must travel.

Pros

- **Reduced latency** – fewer indirections, faster system-call execution, beneficial for high-frequency calls (e.g., gettimeofday).
- **Simplified kernel path** – fewer layers to maintain, easier to reason about critical paths.
- **Potential for micro-optimizations** – hand-tuned assembly can exploit specific processor features.

Cons

- **Portability loss** – assembly is architecture-specific; moving to a new CPU family requires rewriting the routines.
- **Maintainability** – harder to debug and verify compared with high-level C code.
- **Safety** – low-level code increases risk of subtle bugs (e.g., register clobbering) that could compromise the whole system.



Programming Problem: File-Copy Utility (System-Call Interface)

- **Goal:** Write a program that prompts for a source and destination filename, copies the file contents, and performs exhaustive error checking (source existence, read/write failures, permission errors).

- **APIs:** Use either the **POSIX** system calls (open, read, write, close, stat) or the **Windows** API (CreateFile, ReadFile, WriteFile, CloseHandle).
- **Testing:**
 - On **Linux/macOS**, trace the program with strace (Linux) or dtrace (macOS) to verify the exact system-call sequence.
 - On **Windows**, step through the code with a debugger (e.g., Visual Studio) because native tracing tools are unavailable.

The exercise reinforces understanding of how user-level code interacts with the kernel via system calls.

Programming Project: Linux Kernel Modules

Part I – Creating a Simple Module

1. **Source (simple.c)**
 - Includes , , .
 - Defines simple_init() (runs on insmod) and simple_exit() (runs on rmmod).
 - Uses printk(KERN_INFO "...") to log messages; visible with dmesg.
 - Registers entry/exit points with module_init(simple_init); and module_exit(simple_exit);.
 - Supplies metadata macros: MODULE_LICENSE("GPL");, MODULE_DESCRIPTION("Simple Module");, MODULE_AUTHOR("SGG");.
2. **Build**
 - Run make with the provided Makefile; output is simple.ko.
3. **Load / Unload**
 - sudo insmod simple.ko → check with lsmod and dmesg (should see “Loading Module”).
 - sudo rmmod simple → verify removal message via dmesg.
 - Optional: clear kernel log buffer with sudo dmesg -c.

Part II – Using Kernel Linked Lists

- **Data structure:** Circular doubly linked list (struct list_head) from .
- **Example node**

```
struct birthday {
    int day;
    int month;
    int year;
    struct list_head list;    // embeds list pointers
};
```

- **List declaration**

```
static LIST_HEAD(birthday_list);
```

- **Allocation** – kmalloc(sizeof(*person), GFP_KERNEL); (kernel-equivalent of malloc).
- **Initialization** – INIT_LIST_HEAD(&person->list); then list_add_tail(&person->list, &birthday_list);.
- **Traversal** –

```
struct birthday *ptr;
list_for_each_entry(ptr, &birthday_list, list) {
    printk(KERN_INFO "Birthday: %02d-%02d-%04d\n",
        ptr->day, ptr->month, ptr->year);
}
```

- **Deletion** – Use `list_del(&ptr->list);` followed by `kfree(ptr);`.
- **Safe traversal while deleting** – `list_for_each_entry_safe(ptr, tmp, &birthday_list, list)` ensures the next pointer is preserved.

Assignment Tasks

- Create five birthday entries in `simple_init()`.
- Traverse the list, printing each entry to the kernel log.
- In `simple_exit()`, delete all nodes and free the memory, confirming removal with `dmesg`.

The exercise demonstrates kernel-level memory management, linked-list macros, and proper cleanup to avoid leaks.

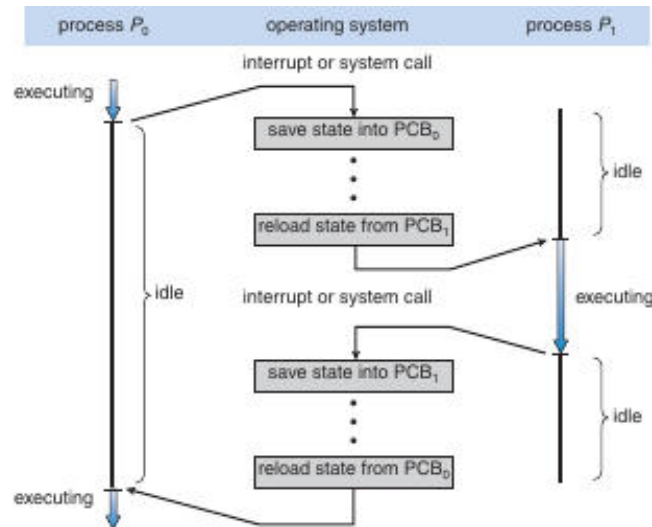


Visual References

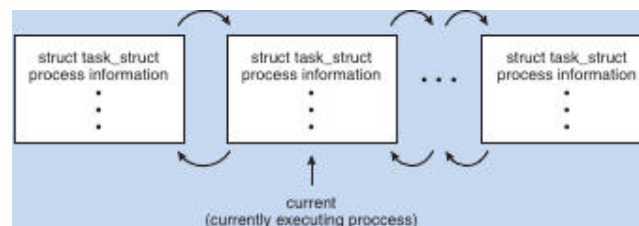
Process State Diagram – illustrates the lifecycle of a process (new → ready → running → waiting → terminated) and the transitions triggered by events such as interrupts or I/O completion.



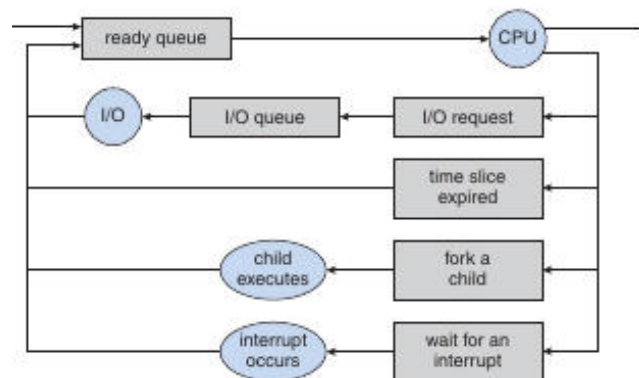
Context-Switch Flowchart – shows how the OS saves the state of the currently running process into its PCB, loads the state of the next process, and resumes execution.



Linux task_struct Representation – the kernel's internal PCB, containing fields for state, scheduling info, parent/children links, file table, and address space.



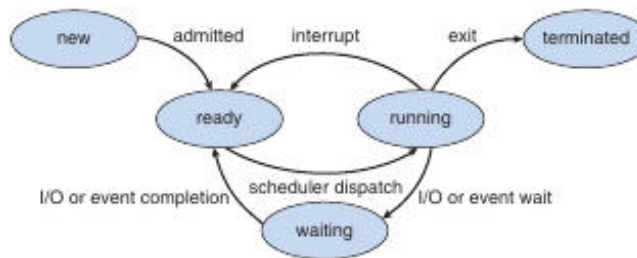
Ready Queue Layout – depicts the ready queue and device-specific I/O queues as linked lists of PCBs awaiting CPU or I/O resources.



Process Scheduling & Queues

Scheduling – the OS mechanism that decides which ready process obtains the CPU next.

- Processes enter the **ready queue** after creation or after completing an I/O request.
- Device queues** hold processes that are blocked waiting for a specific I/O device.
- Arrows in the queuing diagram indicate the flow of processes among these queues.



The diagram shows the ready queue (top) and several I/O queues (bottom). Processes move between them as they are dispatched, request I/O, or are pre-empted.

When a process is dispatched from the ready queue it may:

1. **Issue an I/O request** → moves to the appropriate I/O waiting queue.
2. **Create a child** (e.g., via fork) → the parent may wait for the child's termination.
3. **Be pre-empted by an interrupt** → returns to the ready queue.

The cycle repeats until the process terminates, at which point its PCB and resources are reclaimed.

Scheduler Types

Scheduler	Typical interval	Main purpose	Example decisions
Long-term (job scheduler)	Minutes (or when a process exits)	Controls degree of multiprogramming – how many processes reside in memory	Admit a new batch job from disk, reject if memory would be over-committed
Short-term (CPU scheduler)	Every 10-100 ms	Selects the next ready process for the CPU	Round-robin time slice, priority-based selection
Medium-term (swapper)	Seconds to minutes	Adjusts the multiprogramming level by swapping processes in/out of RAM	Swap out a low-priority, memory-heavy process when RAM is scarce

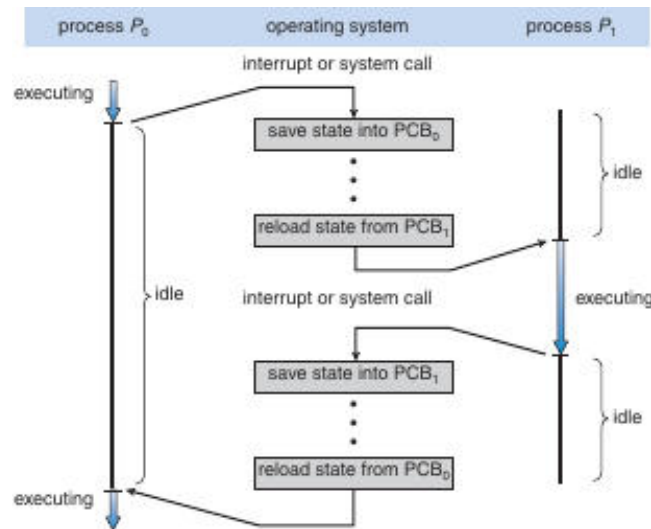
A well-balanced system contains a mix of **CPU-bound** and **I/O-bound** processes; otherwise the ready queue or I/O queues become empty, degrading throughput.

Context Switch & Interrupt Handling

Context switch – saving the state of the currently running process and restoring the state of the next scheduled process.

- The **PCB** stores register values, program counter, and memory-management information.

- Steps:
 1. **Save** registers of the outgoing process into its PCB.
 2. **Update** scheduling data structures (remove from running, insert into ready).
 3. **Load** registers from the incoming process's PCB.
 4. **Resume** execution at the saved program counter.
- Overhead is pure latency; typical switch times are a few milliseconds, but can be reduced by special instructions that save/restore all registers in one operation (e.g., Sun UltraSPARC).



The flowchart outlines the sequence of saving the old context, selecting a new process, and restoring its context.

- **Interrupts** (timer, I/O) trigger the kernel to perform a context switch. The CPU saves the current user state, jumps to the ISR, and after handling the interrupt the scheduler may dispatch a different ready process.

Process Creation

UNIX / Linux

`fork()` – creates a child that is an exact copy of the parent's address space.

- Return value: 0 in the child, child's PID in the parent.
- Typical pattern: child immediately calls `exec()` to replace its memory image with a new program.

```
#include
#include
#include

int main() {
    pid_t pid = fork();
    if (pid < 0) {
```

```

        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        /* child */
        execlp("/bin/ls", "ls", NULL);
    } else {
        /* parent */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}

```

Windows

CreateProcess() – spawns a new process and loads a specified executable directly; the child does **not** inherit the parent's address space.

```

#include
#include

int main(void) {
    STARTUPINFO si = {0};
    PROCESS_INFORMATION pi = {0};
    si.cb = sizeof(si);

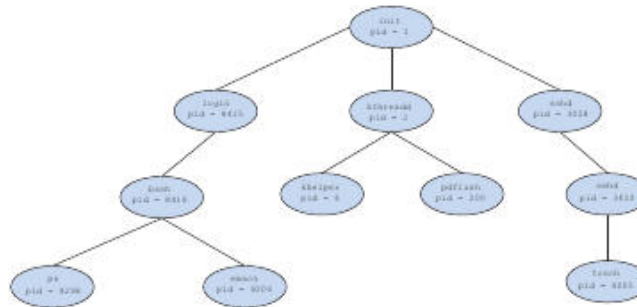
    if (!CreateProcess(
        NULL,
        "C:\\WINDOWS\\system32\\mspaint.exe",
        NULL, NULL, FALSE, 0,
        NULL, NULL, &si, &pi)) {
        fprintf(stderr, "CreateProcess failed\n");
        return -1;
    }

    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete\n");
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}

```

- The parent typically waits on pi.hProcess via WaitForSingleObject().

Process Tree Example



The tree shows *init* (PID 1) as the root, with children such as *kthreadd* and *sshd*, and further descendants like *bash* and *ps*.

- Each process receives a unique **PID** that indexes its PCB and other kernel data structures.

💀 Process Termination, Zombies & Orphans

- A process ends by invoking `exit(status)`.
- The kernel frees **memory, open files, I/O buffers**, but retains a minimal PCB entry (containing the exit status) until the **parent** calls `wait(&status)`.
- If the parent never calls `wait`, the terminated process becomes a **zombie** – it occupies a PID but no other resources.

Situation	Result
Parent calls <code>wait()</code> after child exits	PCB entry removed; PID reclaimed.
Parent exits before calling <code>wait()</code>	Child becomes an orphan ; the init process (PID 1) adopts it and later reaps the zombie.
Parent explicitly kills child (e.g., <code>kill(pid, SIGTERM)</code>)	Child terminates immediately; parent may still need to <code>wait()</code> to avoid a zombie.

- Some systems forbid a child from outliving its parent; if the parent terminates, the OS may terminate all its children automatically (cascading termination).

🤝 Interprocess Communication (IPC) Models

IPC – mechanisms that let cooperating processes exchange data or synchronize actions.

Model	Mechanism	Typical use case
Message passing	send/receive system calls, sockets, pipes	Small control messages, distributed systems
Shared memory	<code>shmget</code> , <code>mmap</code> to map a common region	Large data buffers, high-throughput producer-consumer pipelines

- Message passing avoids the need for explicit synchronization but incurs copy overhead.
- Shared memory offers the fastest data transfer, but requires **mutual exclusion** (mutexes, semaphores) to prevent race conditions.

Both models are often supported simultaneously; a system may use shared memory for bulk data and messages for control information.

Chrome Multiprocess Architecture

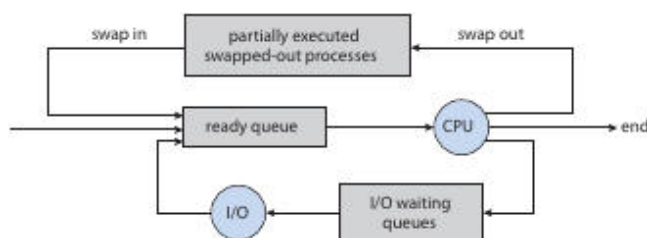
- Chrome isolates different functional components into separate OS processes to improve stability and security.

Process type	Responsibility
Browser process	Handles UI, tab management, disk & network I/O. Only one instance per Chrome session.
Renderer processes	Parse HTML/CSS/JS, layout, paint for each tab. Multiple renderers run concurrently.
Plug-in processes	Host NPAPI/PPAPI plug-ins (e.g., Flash). Each plug-in runs in its own process.

- If a renderer crashes, only the affected tab is lost; the browser and other renderers continue running. This design exemplifies **process isolation** as a practical use of IPC (messages between browser and renderers) and resource containment.

Process Management Flowchart (Swapped-in/out)

The flowchart visualizes the lifecycle of a process as it moves among **swapped-out**, **ready**, **CPU**, and **I/O-waiting** states.



Starting from the swapped-out pool, a process is brought into the ready queue, dispatched to the CPU, and may either complete, be swapped out again, or wait for I/O. I/O-waiting processes reside in a separate queue until the device signals completion.

- The diagram reinforces the earlier description of queues and illustrates how **swapping** (medium-term scheduling) integrates with short-term dispatch.

Summary of Key Concepts

- **Scheduling** coordinates CPU time via long-, short-, and medium-term algorithms.
- **Context switches** save/restore PCB state; their cost depends on hardware support.
- **Process creation** differs between UNIX (fork + exec) and Windows (CreateProcess).

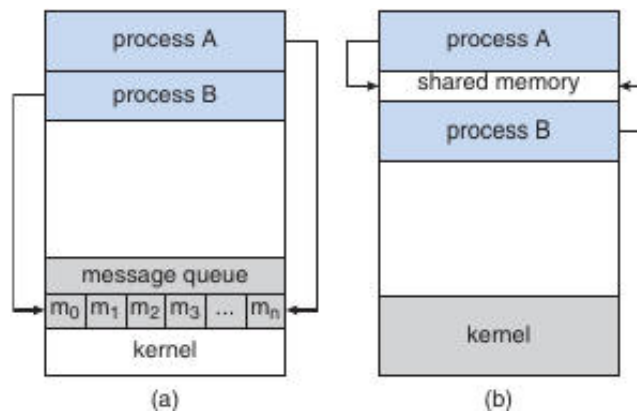
- **Termination** leaves a zombie until the parent reaps it; orphans are adopted by init.
- **IPC** choices (message vs. shared memory) affect performance and complexity.
- **Chrome's multiprocess model** demonstrates real-world use of process isolation for reliability.

Interprocess Communication (IPC) Overview

Interprocess communication – mechanisms that allow cooperating processes to exchange data or synchronize their actions.

Two fundamental IPC models are contrasted:

- **Message-passing** – processes exchange discrete messages via kernel-managed queues.
- **Shared-memory** – processes map a common memory region into their address spaces and read/write directly.



The left side shows processes A and B communicating through a kernel-managed message queue ($m_0, m_1, \dots$). The right side illustrates both processes accessing a common shared-memory area.

Shared-Memory IPC 3.4.1

Shared memory – a region of memory that is mapped into the address spaces of two or more processes, enabling direct data exchange without kernel mediation after the region is established.

Establishing a Shared Region

1. **Create** a shared-memory object (e.g., `shm_open`).
2. **Set size** with `ftruncate`.
3. **Map** the object into each process's address space via `mmap`.

Producer–Consumer Example

The classic **producer-consumer** problem demonstrates coordination over a bounded buffer.

```
#define BUFFER_SIZE 10
typedef struct { /* ... */ } item;
item buffer[BUFFER_SIZE];
int in = 0;    // next free slot
int out = 0;   // next full slot
```

- Buffer empty when `in == out`.
- Buffer full when `((in + 1) % BUFFER_SIZE) == out`.

Producer loop (simplified):

```
while (true) {
    while (((in + 1) % BUFFER_SIZE) == out)    /* wait if full */;
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Consumer loop (simplified):

```
while (true) {
    while (in == out)    /* wait if empty */;
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

The circular array allows up to `BUFFER_SIZE-1` items to be present simultaneously.

Synchronization Issues

- The shared region **does not enforce mutual exclusion**; the programmer must add locks, semaphores, or other primitives to avoid concurrent writes.
- Chapter 5 later covers synchronization techniques for shared-memory environments.



Message-Passing IPC 3.4.2

Message passing – a kernel-provided facility where processes send and receive discrete messages, optionally synchronizing without sharing address space.

Basic Operations

Operation	Description
<code>send(message)</code>	Place a message in a mailbox or queue.
<code>receive(message)</code>	Retrieve the next available message.

Messages may be **fixed-size** (simpler kernel implementation) or **variable-size** (requires more complex handling).

Communication Styles

Style	Naming	Example primitive
Direct	Sender and receiver explicitly name each other.	send(P, msg) / receive(Q, msg)
Indirect	Processes send to a <i>mailbox</i> (or <i>port</i>) identified by a name; any process with access to that mailbox can receive.	send(A, msg) / receive(A, msg)

Synchronization Modes

Mode	Behaviour
Blocking (synchronous)	The caller waits until the operation can complete (e.g., a send blocks until a receiver is ready).
Non-blocking (asynchronous)	The call returns immediately; the caller must later check for completion or handle the failure.

Buffering Strategies

Capacity	Effect on sender/receiver
Zero-capacity (rendezvous)	Sender blocks until the receiver takes the message; no queue is stored.
Bounded (finite \$n\$)	Up to \$n\$ messages can be queued; sender blocks only when the queue is full.
Unbounded	Queue can grow without limit; the sender never blocks.



Naming & Addressing in Message Passing

1. **Direct communication** – processes must know each other's identifiers; a single link exists between any two processes.
2. **Indirect communication** – messages are addressed to a *mailbox* (or *port*) that may be shared among many processes. Mailboxes have unique identifiers (e.g., integer values in POSIX).

Indirect schemes improve modularity because a process can change its internal identifier without affecting other processes; only the mailbox name remains stable.



IPC API Examples

1 POSIX Shared Memory (Section 3.5.1)

```
/* Producer – create and write */
int fd = shm_open("/myshm", O_CREAT | O_RDWR, 0666);
```



```
ftruncate(fd, 4096);
char *ptr = mmap(NULL, 4096, PROT_WRITE, MAP_SHARED, fd, 0);
sprintf(ptr, "Hello World!");
```

```
/* Consumer – open and read */
int fd = shm_open("/myshm", O_RDONLY, 0666);
char *ptr = mmap(NULL, 4096, PROT_READ, MAP_SHARED, fd, 0);
printf("%s\n", ptr);
shm_unlink("/myshm"); // clean up
```

The producer uses `MAP_SHARED` so that modifications are visible to all attached processes.

2 Mach Message Passing (Section 3.5.2)

- **Ports** act as mailboxes.
- `port_allocate()` creates a new mailbox; the creator becomes the *owner* and the only process allowed to receive unless ownership is transferred.
- Messages consist of a fixed-length header (containing sender/receiver port names) plus a variable-length data list.
- The kernel guarantees FIFO ordering **per sender**, but not global ordering across multiple senders.

Sending a message (high-level): `msg_send(port, msg)`

Receiving a message: `msg_receive(port, &msg)`

If a mailbox is full, the sender can:

1. Wait indefinitely.
2. Wait with a timeout.
3. Return immediately with an error.
4. Cache the message locally for later retry (useful for server-type tasks).

3 Windows Advanced Local Procedure Call (ALPC) (Section 3.5.3)

- Uses **ports** (connection ports and communication ports).
- Typical flow:
 1. Server publishes a *connection-port* object.
 2. Client opens the connection port, sending a request.
 3. Server creates a *channel* (pair of private communication ports) and returns a handle to the client.
- Three message-size handling modes:
 - **≤ 256 bytes** – copied through the port's internal queue.
 - **256 bytes – section object** – a shared-memory section is created; the message contains a pointer to the section.
 - **> section size** – the server reads/writes directly into the client's address space via a large shared section.

ALPC is invisible to ordinary Windows API callers; it underlies RPC calls that happen on the same machine.

Design Trade-offs Between Shared Memory and Message Passing

Aspect	Shared Memory	Message Passing
--------	---------------	-----------------

Performance	Very fast after setup (no kernel copy).	May incur kernel copying; Mach mitigates this with virtual-memory mapping.
Complexity	Programmer must handle synchronization, race conditions.	Kernel enforces ordering and optional blocking, simplifying code.
Scalability	Cache-coherency overhead grows with core count; can limit scalability on many-core systems.	Often scales better on multicore / distributed systems; message queues avoid false sharing.
Security	Direct access to another process's memory; requires careful permission management.	Isolation enforced by kernel; sandboxing easier (e.g., Chrome renderer processes).
Portability	Requires explicit setup on each OS (POSIX <code>shm_open</code> , Windows <code>CreateFileMapping</code>).	Generally available via standard IPC APIs (POSIX message queues, Mach ports, Windows ALPC).

Recent research on many-core processors suggests **message passing** may become the preferred IPC mechanism because cache-coherency costs dominate shared-memory performance.

Summary of IPC Concepts

- **Producer-consumer** illustrates both shared-memory (circular buffer) and message-passing (bounded queue) approaches.
- **Direct vs. indirect naming** determines whether processes address each other explicitly or use a shared mailbox.
- **Synchronous vs. asynchronous** defines whether send/receive calls block the caller.
- **Buffer capacity** (zero, bounded, unbounded) governs how many messages can be queued before a sender must wait.
- Real-world systems (Chrome, Mach, Windows) blend these ideas:
 - Chrome isolates tabs in separate sandboxed renderer processes, communicating via message passing.
 - Mach's ports provide flexible, optionally buffered message queues with optional virtual-memory sharing to avoid copies.
 - Windows ALPC combines small-message copying with large-message shared sections to balance speed and simplicity.

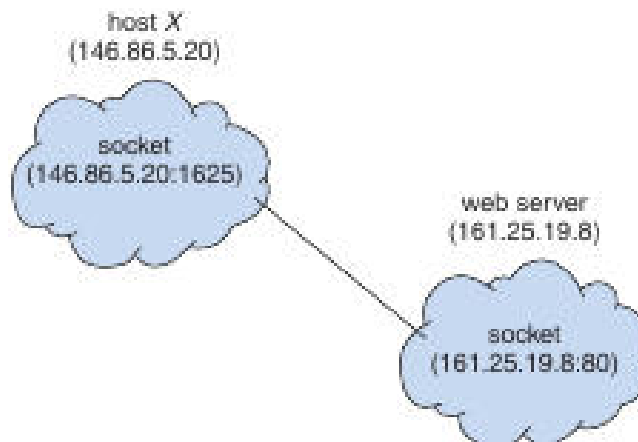
These mechanisms build on the **process management** fundamentals introduced earlier (process states, PCBs, scheduling) and are essential for designing robust, secure, and scalable concurrent applications.

Sockets – endpoint-based network communication (3.6.1)

A **socket** is an endpoint for communication between two processes, identified by an IP address and a port number.

- **Client-server pattern**
 - The **server** creates a listening socket on a well-known port (e.g., 21 for FTP, 80 for HTTP).

- The **client** creates a socket with an arbitrary **ephemeral** port (> 1024) and requests a connection to the server's address-port pair.
- When the server accepts the request, a **pair of connected sockets** is formed: one on the client side, one on the server side.
- **Uniqueness of connections**
 - Each connection is uniquely identified by the four-tuple (*client-IP, client-port, server-IP, server-port*).
 - A second client must use a different ephemeral port; a second connection from the same client must also pick a distinct port.
- **Illustrative network diagram**



The diagram shows host X (IP 146.86.5.20) using an ephemeral port 1625 to reach the web server's well-known HTTP port 80. The connection is represented by the two sockets (146.86.5.20:1625) ↔ (161.25.19.8:80).

- **Socket families in Java**
 - **Socket** – TCP, connection-oriented.
 - **DatagramSocket** – UDP, connectionless.
 - **MulticastSocket** – UDP multicast, delivers a packet to multiple receivers.
- **Simple date-service example (TCP)**
 1. **Server** opens a `ServerSocket` on a chosen port (e.g., 6013).
 2. Calls `accept()` → blocks until a client connects.
 3. Returns a new `Socket` representing the client side of the connection.
 4. Writes the current date to the client with a `PrintWriter`.
 5. Closes the client socket and returns to step 2.

```
// DateServer.java
import java.net.*;
import java.io.*;

public class DateServer {
    public static void main(String[] args) {
        try (ServerSocket listener = new ServerSocket(6013)) {
            while (true) {
                try (Socket client = listener.accept()) {
                    PrintWriter out = new PrintWriter(client.getOutputStream(), true);
                    out.println(new Date());
                }
            }
        }
    }
}
```

```

        out.println(new java.util.Date().toString());
    }
}
} catch (IOException e) {
    System.err.println(e);
}
}
}

```

6. **Client** creates a Socket to the server's address-port, reads the line, prints it, then closes the socket.

```

// DateClient.java
import java.net.*;
import java.io.*;

public class DateClient {
    public static void main(String[] args) {
        try (Socket sock = new Socket("127.0.0.1", 6013);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(sock.getInputStream()))) {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.err.println(e);
        }
    }
}

```

- The loopback address **127.0.0.1** lets a client and server on the same host communicate using the TCP/IP stack, but any reachable host can replace it.
- **Low-level nature of sockets**
 - Sockets transmit a raw byte stream; the application must impose its own protocol (message framing, delimiters, etc.).
 - Higher-level abstractions such as RPC or pipes are built atop sockets to provide structure.

Remote Procedure Calls (RPC) – structured remote service (3.6.2)

An **RPC** abstracts a network request as a local-like procedure call: the client invokes a stub as if the procedure were local, while the RPC runtime packages the arguments, sends them to a remote daemon, executes the real procedure, and returns the result.

Core components

Component	Role
-----------	------

Client stub	Marshals (packs) the arguments, attaches the procedure identifier, and sends the request to the server's RPC daemon.
Server stub	Receives the request, unmarshals (unpacks) arguments, invokes the actual procedure, marshals any return value, and replies.
RPC daemon	Listens on a well-known port, dispatches incoming requests to the appropriate server stub.
Matchmaker (rendezvous)	Optional service that maps a logical RPC name to a concrete port number at runtime.

Data representation

- Different architectures may use **big-endian** or **little-endian** byte order.
- RPC systems adopt a **machine-independent format** (e.g., **XDR – External Data Representation**) for all transmitted data.
- **Marshalling** = converting native data to XDR before transmission; **unmarshalling** = converting XDR back to native form on receipt.

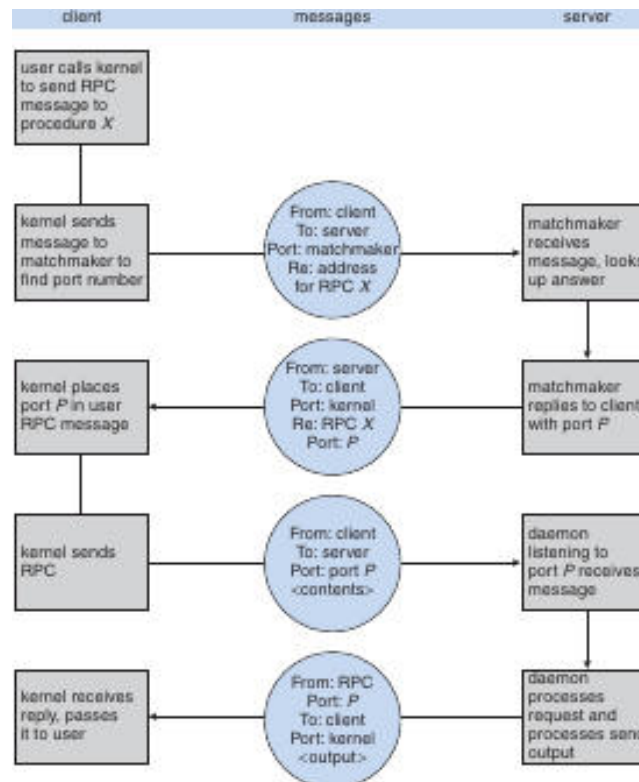
Call semantics

Semantic	Guarantees	Typical implementation
At-most-once	The server executes the procedure zero or one time.	Server tracks timestamps (or unique IDs) of processed messages; duplicates are discarded.
Exactly-once	The server executes the procedure exactly one time; the client is assured of completion.	Combination of at-most-once detection plus an acknowledgment (ACK) from the server. The client retransmits until it receives the ACK.

Binding the client to the server

1. **Static (pre-determined) binding** – The client code contains the fixed port number of the desired service. No lookup required, but the service cannot change its listening port without recompiling clients.
2. **Dynamic binding via a rendezvous daemon** –
 - The client sends a request to a well-known matchmaker port, asking for the current port of a named service.
 - The matchmaker replies with the service's actual port, and the client proceeds to use that port for subsequent RPC calls.
 - This adds an initial lookup overhead but permits flexible port allocation and service migration.

Example interaction (illustrated)



The flowchart shows the client invoking a stub, the kernel forwarding the request to a matchmaker (if needed), the RPC daemon receiving the message, executing the remote procedure, and sending the reply back through the kernel to the client stub.

Typical use cases

- **Distributed file systems** – file-operation requests are implemented as RPCs to remote file-service daemons.
- **System administration services** – e.g., a daemon listening on port 3027 that returns the list of logged-in users.
- **Cross-platform libraries** – Microsoft's **MIDL** generates stub code for Windows RPC, handling marshalling, unmarshalling, and network transport.



Pipes – another higher-level IPC mechanism

A **pipe** provides a *unidirectional byte stream* between a producer and a consumer process, typically created by the operating system when one process invokes `pipe()` (POSIX) or `CreatePipe` (Windows).

- Pipes are **local** (same host) and inherit the **stream semantics** of sockets, but the OS supplies the buffer and handles synchronization automatically.
- They are frequently used to connect the standard output of one process to the standard input of another (e.g., `ls | grep foo`).
- Unlike sockets, pipes have **no addressing**; the two ends are linked directly at creation time.

Comparative overview of client-server communication mechanisms

Mechanism	Scope	Transport model	Data structuring	Typical APIs / classes	Typical use
Sockets	Local or network	Byte-stream (TCP) or datagram (UDP)	Application-defined protocol	Socket, DatagramSocket, MulticastSocket (Java)	General client-server services (HTTP, FTP, custom protocols)
RPC	Network (often distributed)	Message-based over sockets (or other transports)	XDR / language-specific marshalling	MIDL (Windows), rpcgen (Unix)	Distributed services (file systems, authentication, remote management)
Pipes	Local only	Unidirectional stream	Implicit (byte stream)	pipe() (POSIX), CreatePipe (Windows)	Shell pipelines, parent-child communication
Shared memory	Local only	Direct memory mapping	Requires explicit synchronization	shm_open, mmap (POSIX)	High-throughput producer-consumer multimedia buffers

- **Performance** – Shared memory > Pipes ≈ Sockets > RPC (extra marshalling overhead).
- **Complexity** – RPC adds the most abstraction (stubs, marshalling, semantics), while sockets are lowest-level but require the programmer to define a protocol.
- **Portability** – RPC frameworks hide platform differences; sockets and pipes are available on virtually all OSes but need explicit handling of endianness and data formats.

Key takeaways (linked to earlier sections)

- The **client-server model** built on sockets mirrors the **process-state transitions** discussed in the process-management chapter: a server blocks in `accept()`, a client blocks in `connect()`, and both resume when the TCP three-way handshake completes.
- **RPC** extends the IPC concepts from Section 3.4 (shared memory, message passing) by adding **procedure abstraction** and **semantic guarantees** (at-most-once, exactly-once).
- **Pipes** illustrate a specialized case of **interprocess communication** that the OS implements directly, avoiding the need for explicit networking code.

These mechanisms together provide the toolbox for building robust, scalable client-server applications across the diverse operating environments covered throughout the textbook.